



Báo cáo bản đi in - dfsagdfa

cơ sở an toàn bảo mật thông tin (Học Viện Kỹ Thuật Mật Mã)



Scan to open on Studeersnel

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



Báo cáo chuyên đề cơ sở

Đề tài:

**NGHIÊN CỨU VÀ XÂY DỰNG MÔ HÌNH PHÁT HIỆN MÃ ĐỘC
DỰA TRÊN SIÊU DỮ LIỆU (METADATA) CỦA TẬP TIN PE**

Sinh viên thực hiện:

1. **Nguyễn Đức Hùng – AT190223**
2. **Lê Minh Đăng – AT190109**
3. **Tô Duy Anh – AT190205**

Nhóm: 7

Giảng viên hướng dẫn:

ThS. Thái Thị Thanh Vân

Khoa Công nghệ thông tin – Học viện Kỹ thuật mật mã

Hà Nội, 2025

BÁO CÁO CHUYÊN ĐỀ CƠ SỞ

**NGHIÊN CỨU VÀ XÂY DỰNG MÔ HÌNH PHÁT
HIỆN MÃ ĐỘC DỰA TRÊN SIÊU DỮ LIỆU
(METADATA) CỦA TẬP TIN PE**

Nhận xét của giảng viên hướng dẫn:

.....

.....

.....

.....

.....

.....

.....

.....

Điểm chuyên cần:

Điểm báo cáo:

Xác nhận của giảng viên hướng dẫn

MỤC LỤC

DANH MỤC KÝ HIỆU VÀ VIẾT TẮT	1
DANH MỤC HÌNH VẼ.....	2
DANH MỤC BẢNG.....	3
LỜI CẢM ƠN	4
LỜI NÓI ĐẦU	5
CHƯƠNG 1. TỔNG QUAN VỀ MÃ ĐỘC VÀ PHÂN TÍCH MÃ ĐỘC	6
1.1 Tổng quan về mã độc.....	6
1.1.1 Khái niệm mã độc.....	6
1.1.2 Mục đích của mã độc.....	7
1.1.3 Tình hình mã độc tại Việt Nam và trên thế giới	8
1.1.4 Phân loại mã độc.....	11
1.2. Kỹ thuật phân tích mã độc	19
1.2.1 Phân tích động	20
1.2.2 Phân tích tĩnh	21
1.3 Kết luận chương 1	23
CHƯƠNG 2. PHÁT HIỆN MÃ ĐỘC DỰA TRÊN METADATA CỦA TẬP TIN	
24	
2.1 Khái niệm về meta-data	24
2.1.1 Định nghĩa.....	24
2.1.2 Cách sử dụng siêu dữ liệu.....	27

2.1.3 Mô tả dữ liệu.....	28
2.2 Cấu trúc tệp tin thực thi trên hệ điều hành Windows	30
2.2.1 Khái niệm PE File.....	30
2.2.2 Cấu trúc cơ bản của một PE File	31
2.2.3 DOS MZ Header.....	33
2.2.4 DOS STUB	35
2.2.5 PE Header	36
2.2.6 Data Directory.....	39
2.2.7 Section Table.....	43
2.3 Các mô hình học máy	45
2.3.1. Mô hình cây quyết định Decision Tree.....	45
2.3.2 Mô hình Rừng ngẫu nhiên Random Forest	46
2.3.3. Mô hình Naive Bayes	48
CHƯƠNG 3. TRIỂN KHAI THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ .	50
3.1 Triển khai giải pháp.....	50
3.1.1 Giới thiệu giải pháp	50
3.1.2 Các bước triển khai.....	51
3.2 Cài đặt thử nghiệm.....	56
3.2.1 Môi trường cài đặt	56
3.2.2 Thử nghiệm.....	57
3.3 Đánh giá kết quả	58
3.4 Kết luận chương 3.....	62

KẾT LUẬN.....	63
TÀI LIỆU THAM KHẢO.....	65

DANH MỤC KÝ HIỆU VÀ VIẾT TẮT

RT	Random Forest
DT	Decision Tree
ANN	Artificial Neural Network
NB	Naive Bayes
RVA	Relative Virtual Address
DDOS	Distributed Denial Of Service
API	Application Programming Interface
DNS	Domain Name System
AI	Artificial Intel
PE	Portable Executable
IDA	Interactive DisAssembler
TGHL	Thời gian huấn luyện
TGTN	Thời gian thử nghiệm

DANH MỤC HÌNH VẼ

Hình 1.1: Ví dụ về mã độc	7
Hình 1.2: Biểu đồ tổng số phần mềm độc hại từ 2008 đến 2024.....	9
Hình 1.3: Biểu đồ tấn công mạng các trang, cổng TTĐT tại Việt Nam qua các năm	11
Hình 1.4: Chức năng Pseudocode-A tạo mã giả C của IDA pro.....	21
Hình 1.5: Chương trình được phân tích bởi IDA pro.....	22
Hình 2.1: Cấu trúc tập tin thực thi PE.....	33
Hình 2.2: Cấu trúc Image_Dos_Header.....	34
Hình 2.3: Giá trị của lfanew trên hex dump.....	35
Hình 2.4: Giá trị của DOS STUB trên hexdump	36
Hình 2.5: Cấu trúc Image_NT_Header	36
Hình 2.6: Cấu trúc Image_File_Header	37
Hình 2.7: Cấu trúc Image_Optional_Header32	38
Hình 2.8: Cấu trúc Image_Data_Directory	40
Hình 2.9: Cấu trúc dữ liệu được định nghĩa bởi Data Directory	42
Hình 2.10: Cấu trúc Image_Section_Header	43
Hình 3.1: Bản ghi dữ liệu sau khi thực hiện trích xuất và gán nhãn.....	53
Hình 3.2: Đặc trưng e_lfanew	54
Hình 3.3: Đặc trưng e_magic và đặc trưng e_crlc	54
Hình 3.4: Đặc trưng NumberOfSection và Characteristics.....	55
Hình 3.5: Hai đặc trưng SizeOfStackReverse và MinorLinkerVersion.....	55

DANH MỤC BẢNG

Bảng 1. Kết quả phân lớp với Dataset_selected.csv gồm 53 đặc trưng.....	59
Bảng 2. Kết quả phân lớp với Dataset_selected.csv gồm 14 đặc trưng.....	61

LỜI CẢM ƠN

Chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến cô Thái Thị Thanh Vân – người đã tận tình giảng dạy, hướng dẫn và đồng hành cùng chúng em trong suốt quá trình thực hiện đề tài “Nghiên cứu và xây dựng mô hình phát hiện mã độc dựa trên siêu dữ liệu (metadata) của tập tin PE”.

Sự chỉ dẫn và định hướng quý báu từ cô không chỉ giúp chúng em nắm vững kiến thức chuyên môn, mà còn tạo điều kiện thuận lợi để nhóm hoàn thành đề tài một cách trọn vẹn. Cô đã luôn kiên nhẫn giải đáp các thắc mắc, đưa ra những nhận xét xác đáng và chỉ ra các hướng nghiên cứu phù hợp, giúp chúng em có thể cải thiện và nâng cao chất lượng của bài làm.

Trong quá trình thực hiện, chúng em cũng nhận thấy đề tài vẫn còn những hạn chế và thiếu sót nhất định. Vì vậy, chúng em rất mong nhận được sự góp ý và đánh giá từ cô để tiếp tục hoàn thiện và rút ra thêm nhiều bài học kinh nghiệm quý giá cho các nghiên cứu sau này.

Một lần nữa, chúng em xin trân trọng cảm ơn cô vì sự tận tâm, nhiệt huyết và sự hỗ trợ hết lòng trong suốt quá trình học tập và nghiên cứu. Những kiến thức, kinh nghiệm và sự động viên từ cô là nguồn động lực to lớn giúp chúng em vượt qua khó khăn và tự tin hơn trên con đường học thuật.

Chúng em xin kính chúc cô luôn dồi dào sức khỏe, nhiều niềm vui và thành công trong sự nghiệp giảng dạy.

Nhóm sinh viên thực hiện đề tài

LỜI NÓI ĐẦU

Trong bối cảnh số lượng phần mềm độc hại (malware) ngày càng gia tăng và trở nên tinh vi, việc phát triển các phương pháp phát hiện mã độc hiệu quả là một yêu cầu cấp thiết trong lĩnh vực an ninh mạng. Bên cạnh các kỹ thuật phân tích hành vi và giám sát thời gian thực, việc khai thác các đặc trưng tĩnh từ tệp thực thi - đặc biệt là thông tin siêu dữ liệu trong PE (Portable Executable) Header - đang nổi lên như một hướng đi đầy tiềm năng, với tối ưu là nhanh chóng, ít tốn tài nguyên và phù hợp với quy mô lớn.

Báo cáo này được thực hiện nhằm nghiên cứu và đánh giá khả năng phát hiện phần mềm độc hại dựa trên các đặc trưng và huấn luyện các mô hình học máy khác nhau (Naive Bayes, Decision Tree và Random Forest), nêu ra các ưu nhược điểm của từng mô hình, bọn em hướng đến việc tìm ra mô hình cũng như cách lựa chọn các đặc trưng sao cho có độ chính xác cao và tối ưu thời gian nhất.

Hy vọng rằng kết quả của báo cáo này không chỉ góp phần khẳng định vai trò của phân tích tĩnh trong phòng chống mã độc, mà còn là tài liệu tham khảo hữu ích cho những nghiên cứu và ứng dụng tiếp theo trong lĩnh vực bảo mật hệ thống. Nhóm em xin chân thành cảm ơn.

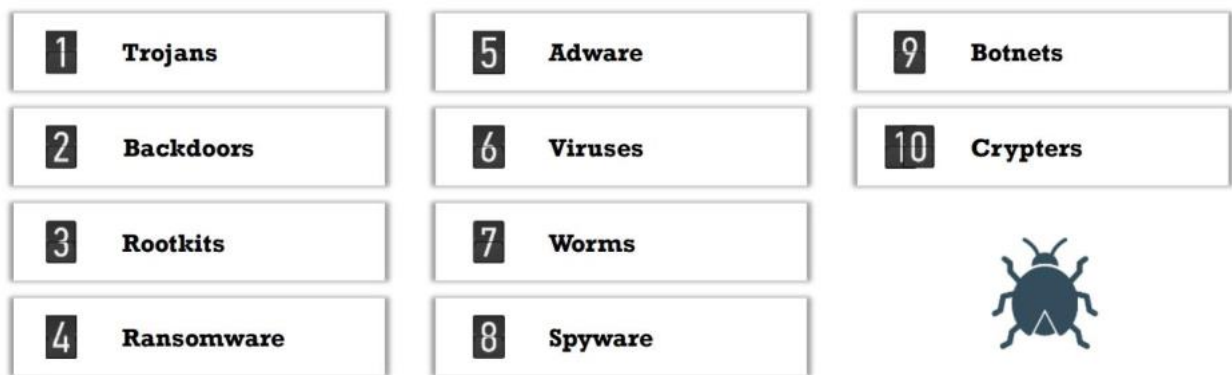
CHƯƠNG 1. TỔNG QUAN VỀ MÃ ĐỘC VÀ PHÂN TÍCH MÃ ĐỘC

1.1 Tổng quan về mã độc

1.1.1 Khái niệm mã độc

Ngay từ những ngày đầu của kỷ nguyên máy tính, mã độc đã xuất hiện. Một trong những ví dụ đầu tiên được ghi nhận là loại sâu máy tính có tên Creeper Worm vào những năm 1970. Đây là một chương trình tự sao chép, hoạt động như một thử nghiệm, có khả năng lan truyền tới các hệ thống từ xa và hiển thị thông điệp: "I'm the Creeper, catch me if you can". Đến đầu thập niên 80, với sự xuất hiện của Elk Cloner – một loại virus nhắm đến các máy tính Apple II – ngành công nghiệp bảo mật bắt đầu hình thành và lớn mạnh, mở ra một cuộc chiến dai dẳng chống lại phần mềm độc hại, cuộc chiến vẫn chưa có hồi kết.

Theo quan điểm của Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST – *National Institute of Standards and Technology*), trong lĩnh vực định nghĩa và phân loại “virus máy tính”, mã độc (*malware*) được hiểu là *một chương trình được chèn vào hệ thống một cách bí mật nhằm gây tổn hại đến tính bí mật, tính toàn vẹn hoặc tính sẵn sàng của hệ thống*. Theo định nghĩa này, mã độc bao gồm nhiều thể loại khác nhau, mà tại Việt Nam thường gọi chung là virus máy tính, chẳng hạn như *worm, trojan, spyware,...* Thậm chí còn bao gồm cả các công cụ tấn công hệ thống thường được tin tặc sử dụng như *backdoor, rootkit, keylogger*.



Hình 1.1: Ví dụ về mã độc

Theo thời gian, các loại mã độc ngày càng trở nên tinh vi và khó phát hiện hơn nhờ vào các kỹ thuật lây lan phức tạp, khả năng ẩn mình cao và cách thức tấn công đa dạng. Ranh giới giữa các loại mã độc cũng dần mờ nhạt, khi mà nhiều chủng mã độc hiện đại có sự kết hợp giữa các cơ chế khác nhau nhằm tối đa hóa hiệu quả tấn công.

1.1.2 Mục đích của mã độc

Ban đầu, mã độc (malware) được tạo ra với mục đích chính là phục vụ cho các thử nghiệm kỹ thuật, hoặc đơn thuần là nhằm phá hoại một hệ thống máy tính đơn lẻ, thường là để chứng minh khả năng của người lập trình hoặc kiểm tra lỗ hổng bảo mật. Tuy nhiên, cùng với sự phát triển mạnh mẽ của công nghệ thông tin và Internet, mã độc đã nhanh chóng trở thành một công cụ nguy hiểm được khai thác với nhiều mục đích tinh vi và quy mô lớn hơn.

Mục đích phổ biến nhất của mã độc hiện nay là đánh cắp thông tin cá nhân và dữ liệu nhạy cảm của người dùng. Các loại thông tin bị nhắm đến bao gồm tài khoản đăng nhập, mật khẩu, dữ liệu ngân hàng, thông tin tài chính, thông tin liên quan đến doanh nghiệp, và thậm chí là dữ liệu thuộc lĩnh vực quân sự, quốc phòng. Tin tặc sử dụng mã độc để thu thập những dữ liệu này nhằm phục vụ cho hoạt động lừa đảo,

bán dữ liệu trên chợ đen, hoặc thực hiện các hành vi tống tiền và phá hoại. Một số loại mã độc như spyware, keylogger hay trojan được thiết kế đặc biệt để theo dõi hoạt động người dùng và gửi dữ liệu về cho kẻ tấn công mà không bị phát hiện.

Ngoài ra, mã độc còn được sử dụng như một công cụ trong các chiến dịch tấn công chính trị và gián điệp mạng. Nhiều cuộc tấn công mạng có chủ đích đã nhắm vào các cơ quan chính phủ, tổ chức quốc tế hoặc các công ty công nghệ lớn nhằm làm gián đoạn hoạt động, phá hoại hệ thống, hoặc đánh cắp thông tin được bảo vệ nghiêm ngặt. Điển hình là các cuộc tấn công từ chối dịch vụ phân tán (DDoS) sử dụng mạng máy tính ma (botnet) để khiến hàng loạt hệ thống không thể truy cập được. Năm 2023, Hàn Quốc ghi nhận một cuộc tấn công mạng quy mô lớn khiến hơn 30 trang thông tin chính thức của chính phủ bị tê liệt trong nhiều giờ. Tại Việt Nam, vào ngày 14/6/2021, trang báo điện tử của Đài Tiếng nói Việt Nam (VOV) cũng từng bị tấn công DDoS, buộc phải tạm ngừng hoạt động để khắc phục sự cố.

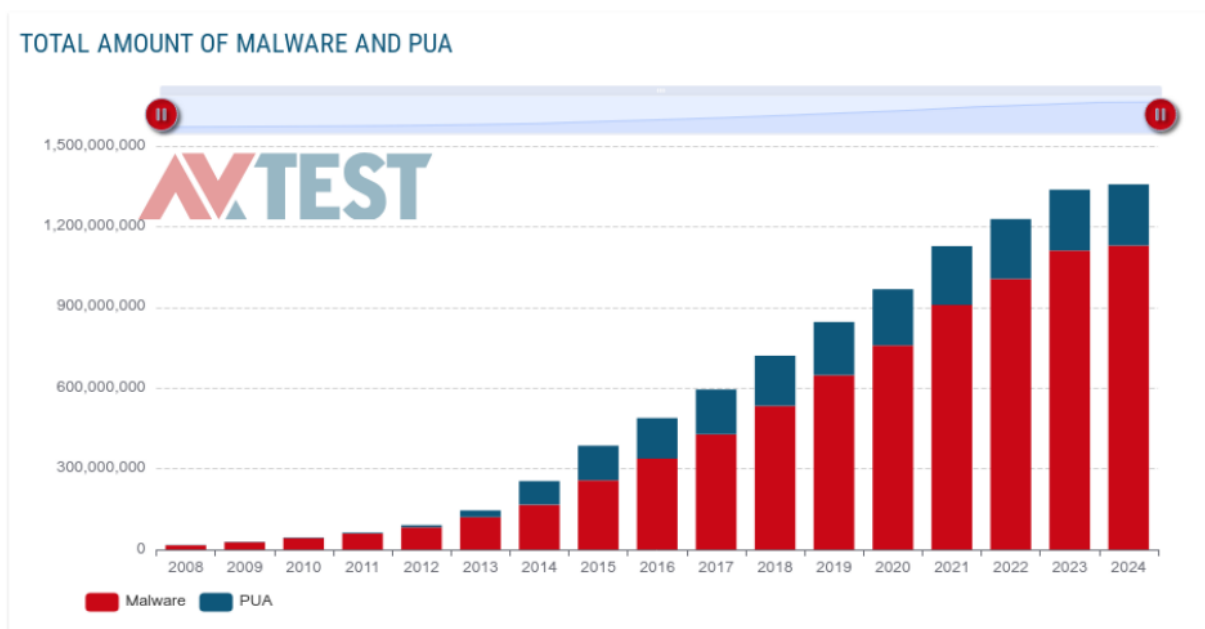
Bên cạnh đó, mã độc còn được sử dụng trong các hoạt động phá hoại có chủ đích như cài đặt phần mềm tống tiền (ransomware), mã hóa toàn bộ dữ liệu của nạn nhân và yêu cầu một khoản tiền chuộc để khôi phục. Các hình thức tấn công như vậy đang gia tăng nhanh chóng trong những năm gần đây, gây thiệt hại nghiêm trọng về kinh tế, an ninh thông tin và uy tín của tổ chức bị ảnh hưởng.

Tóm lại, mục đích của mã độc ngày càng đa dạng và phức tạp, không chỉ dừng lại ở việc phá hoại mà còn phục vụ cho các mục tiêu kinh tế, chính trị, gián điệp và chiến tranh mạng trong thời đại số.

1.1.3 Tình hình mã độc tại Việt Nam và trên thế giới

Kể từ khi những phần mềm độc hại đầu tiên xuất hiện, số lượng mã độc đã không ngừng gia tăng với tốc độ chóng mặt. Theo báo cáo từ tổ chức nghiên cứu độc

lập về an toàn thông tin AV-TEST, tính đến đầu năm 2024, cơ sở dữ liệu của họ đã ghi nhận hơn 1,5 tỷ phần mềm độc hại được phát hiện và phân loại. Trung bình, mỗi ngày có khoảng 450.000 phần mềm độc hại mới xuất hiện, trong đó phần lớn nhắm đến hệ điều hành Windows. Sự gia tăng liên tục này đặt ra nhiều thách thức nghiêm trọng về an toàn thông tin và an ninh mạng cho người dùng Internet trên toàn thế giới, từ cá nhân đến tổ chức, doanh nghiệp.



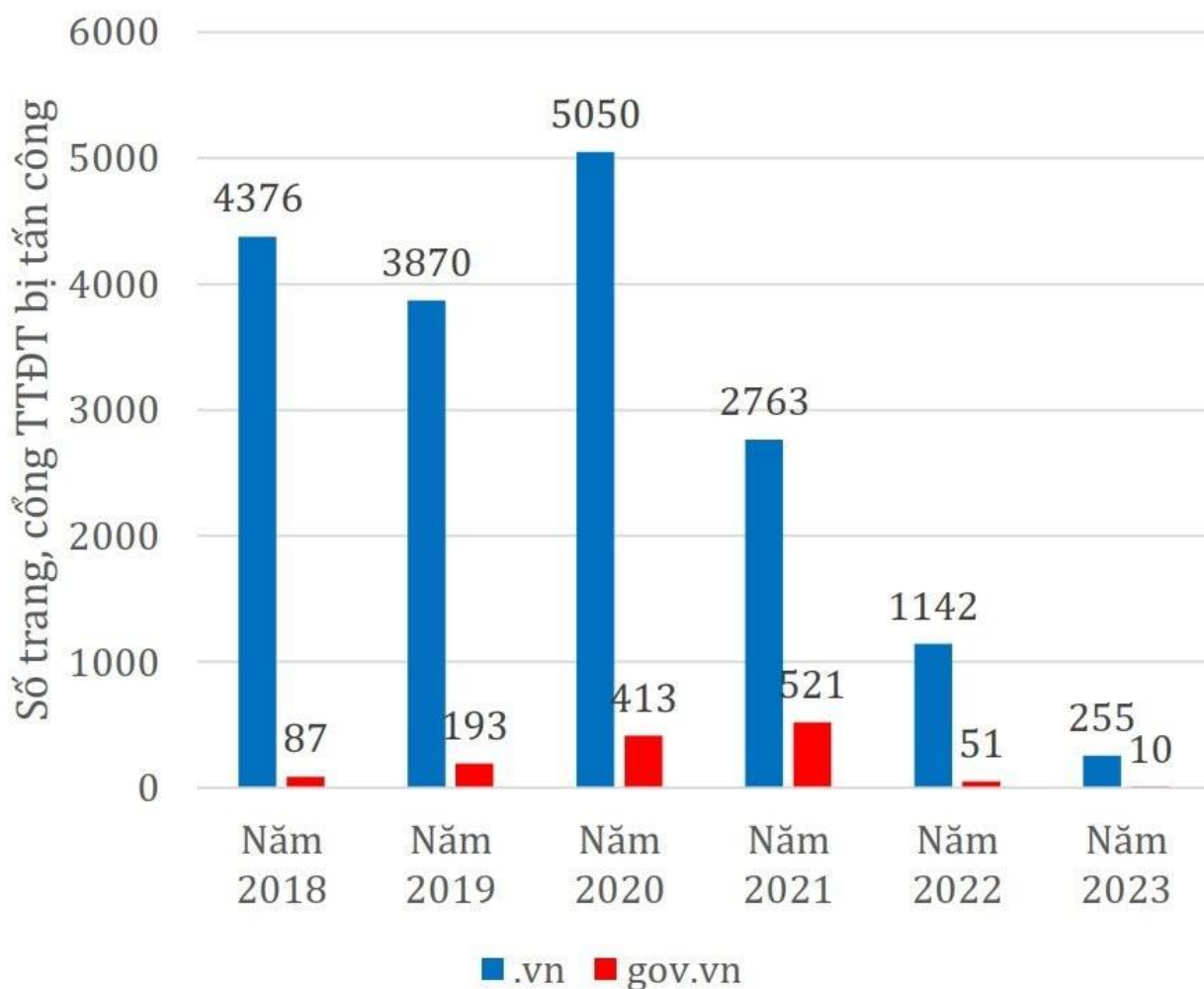
Nguồn: av-atlas.org

Hình 1.2: Biểu đồ tổng số phần mềm độc hại từ 2008 đến 2024

Chủng loại mã độc ngày càng đa dạng và phong phú cả về hành vi lẫn mục đích phát tán. Các lĩnh vực mà mã độc nhắm đến không chỉ dừng lại ở kinh tế, chính trị, tôn giáo mà còn lan rộng sang các hệ thống hạ tầng thiết yếu và cả đời sống cá nhân. Những năm gần đây, nhiều chiến dịch tấn công quy mô lớn như ransomware, botnet, DDoS đã liên tục gây rúng động toàn cầu.

Tại Việt Nam, các hình thức phát tán phần mềm chứa mã độc vào cơ quan, tổ chức và doanh nghiệp ngày càng phổ biến, đặc biệt thông qua email giả mạo chứa tệp văn bản đính kèm độc hại. Những tập tin này thường khai thác lỗ hổng trong bộ phần mềm văn phòng như Word, Excel hay PowerPoint để kích hoạt mã độc. Sau khi xâm nhập, mã độc âm thầm tạo cửa hậu (Backdoor), cho phép kẻ tấn công kiểm soát máy tính nạn nhân từ xa, đánh cắp dữ liệu, theo dõi thao tác bàn phím, ghi hình, chụp màn hình hoặc phát tán tiếp mã độc khác.

Theo số liệu từ AV-TEST – tổ chức nghiên cứu độc lập về an toàn thông tin, tính đến năm 2024, đã có hơn 1,5 tỷ phần mềm độc hại được phát hiện và ghi nhận trên toàn thế giới. Trung bình mỗi ngày có hơn 450.000 mẫu mã độc mới xuất hiện. Riêng tại khu vực châu Âu, năm 2024 ghi nhận 4.295 vụ tấn công mạng nghiêm trọng, trong đó có hơn 3.000 vụ là tấn công từ chối dịch vụ phân tán (DDoS) và hơn 800 vụ là tấn công bằng mã độc tống tiền (ransomware). Những con số này phản ánh rõ rệt mức độ nghiêm trọng và tốc độ lan rộng của mã độc trên phạm vi toàn cầu, trong đó Việt Nam vẫn là một điểm nóng về phát tán và nhiễm mã độc trong khu vực.



Hình 1.3: Biểu đồ tấn công mạng các trang, cổng TTĐT tại Việt Nam qua các năm

1.1.4 Phân loại mã độc

1.1.4.1 Các loại mã độc phổ biến

Mã độc hại (Malware) được hiểu là một phần mềm được cài đặt hoặc xâm nhập vào hệ thống một cách bí mật nhằm phá hoại hoặc làm ảnh hưởng đến tính bí mật, toàn vẹn và khả năng sẵn sàng của hệ thống thông tin.

Mọi người hay bị nhầm lẫn với 1 khái niệm khác là virus máy tính. Thực tế, virus máy tính chỉ là 1 phần nhỏ trong khái niệm mã độc. Virus máy tính hiểu đơn

thuần cũng là một dạng mã độc nhưng sự khác biệt ở chỗ virus máy tính có khả năng tự lây lan. Mã độc được phân thành nhiều loại tùy theo chức năng, cách thức lây nhiễm, phá hoại. Trong đó có thể kể đến các loại phổ biến như: sâu máy tính (worm), trojan, phần mềm gián điệp (spyware), phần mềm quảng cáo (adware)... Bên cạnh đó còn có các công cụ tấn công hệ thống mà tin tặc thường sử dụng như cửa hậu (backdoor), bộ công cụ che giấu (rootkit), trình ghi lại thao tác bàn phím (keylogger), mạng máy tính ma (botnet), và đặc biệt là các loại mã độc tống tiền – ransomware – có khả năng mã hóa dữ liệu để đòi tiền chuộc.

Để có thể nhận diện và xử lý hiệu quả các mối đe dọa này, chúng ta cần tìm hiểu rõ về cơ chế phân loại mã độc, mục tiêu của việc phân loại, lý do cần thiết phải phân loại cũng như xác định rõ các đối tượng cần được phát hiện và bảo vệ trước sự xâm nhập của phần mềm độc hại.

a. Virus máy tính

Trong lĩnh vực khoa học máy tính, virus máy tính (hay còn gọi là virus) là các chương trình hoặc đoạn mã độc hại được thiết kế để tự nhân bản và lây lan sang các đối tượng khác như tập tin, ổ đĩa hoặc hệ thống máy tính khác. Ban đầu, virus thường được tạo ra bởi các lập trình viên nhằm chứng minh khả năng kỹ thuật của họ, với mục đích phá hoại như làm hỏng các chương trình, xóa dữ liệu hoặc gây hại cho phần cứng. Tuy nhiên, virus hiện đại ngày nay không chỉ dừng lại ở những hành vi phá hoại đơn thuần, mà chúng còn nhắm đến mục tiêu đánh cắp thông tin nhạy cảm như số thẻ tín dụng, tài khoản ngân hàng, hoặc tạo ra các "cửa hậu" (backdoor) cho phép tin tặc xâm nhập và kiểm soát hệ thống.

Khoảng 90% virus được phát hiện hiện nay nhắm vào các hệ thống sử dụng hệ điều hành Windows, bởi đây là hệ điều hành phổ biến nhất trên thế giới. Sự phổ biến

này đã khiến Windows trở thành mục tiêu chính của các tin tặc. Bên cạnh các loại virus truyền thống, hiện nay còn xuất hiện các biến thể tinh vi hơn, bao gồm virus đa hình (polymorphic) và siêu đa hình (meta-polymorphic).

Virus đa hình khác với các virus thông thường ở chỗ nó có khả năng tự thay đổi mã lệnh của mình sau mỗi lần lây nhiễm. Điều này tạo ra những phiên bản mã khác nhau, giúp virus tránh được sự phát hiện của các phần mềm diệt virus truyền thống. Virus siêu đa hình là một phiên bản nâng cấp của virus đa hình, kết hợp nhiều kỹ thuật đa hình để tạo ra các thể hệ virus con từ F1 đến Fn, với mỗi thể hệ có những đặc điểm mã hóa khác biệt. Sự lai tạo này khiến virus càng trở nên phức tạp và khó bị phát hiện hơn. Các phần mềm diệt virus không được trang bị cơ chế quét sâu có thể gặp khó khăn trong việc nhận diện chúng. Ví dụ điển hình của virus siêu đa hình là Vektor và Sality, những biến thể này tạo ra thách thức lớn trong việc bảo mật hệ thống và yêu cầu các biện pháp phòng chống tiên tiến hơn.

b. Trojan horse

Trojan Horse, hay còn gọi là Trojan, là một loại phần mềm độc hại có khả năng điều khiển máy tính qua các chương trình cụ thể. Khác với các loại virus thông thường, Trojan không tự nhân bản hay lây lan sang các tập tin khác. Thay vào đó, nó lừa người dùng tải về thông qua các phần mềm nguy trang, rồi từ đó tạo kết nối với hệ thống bị xâm nhập. Kẻ tấn công sau đó có thể thực hiện các hành vi phá hoại như đánh cắp dữ liệu, điều khiển máy tính từ xa, hoặc thậm chí làm hỏng hệ thống.

Trojan có thể sử dụng một trong ba phương thức sau để gây hại:

- Tiếp tục thực thi chức năng của chương trình gốc mà nó bám vào, nhưng đồng thời thực hiện các hoạt động độc hại một cách âm thầm, ví dụ như

gửi một trò chơi để dụ người dùng tải về, trong khi âm thầm cài đặt chương trình thu thập thông tin người dùng.

- Tiếp tục thực thi chức năng của chương trình gốc, nhưng sửa đổi hoặc thay đổi một số tính năng của chương trình đó để gây tổn hại, ví dụ như tạo ra một cửa sổ đăng nhập giả để lấy cắp mật khẩu của người dùng, hoặc ẩn giấu các hoạt động phá hoại khác như tắt các thông báo hệ thống.
- Thực thi chương trình gây hại dưới vỏ bọc của một ứng dụng vô hại, chẳng hạn như một trò chơi hoặc công cụ trên mạng. Khi người dùng kích hoạt tệp tin này, dữ liệu trên máy tính có thể bị xóa hoặc bị chiếm quyền điều khiển ngay lập tức.

Các đoạn mã Trojan thường được “giấu” trong các loại virus khác hoặc trong các phần mềm hợp pháp để xâm nhập vào hệ thống một cách bí mật. Sau khi xâm nhập, chúng sẽ thực hiện các hành động như ăn cắp thông tin cá nhân, mật khẩu hoặc điều khiển máy tính nạn nhân từ xa. Điểm đặc biệt của Trojan là nó không tự lây lan mà phải dựa vào phần mềm khác để phát tán.

Dựa vào cách thức hoạt động, Trojan có thể được phân chia thành các loại sau: Backdoor (cửa hậu), Adware (phần mềm quảng cáo) và Spyware (phần mềm gián điệp).

c. Worm

Sâu máy tính là một dạng phần mềm độc hại có khả năng tự nhân bản và lây lan qua mạng Internet mà không cần sự tương tác của người dùng. Khác với virus, sâu không cần gắn kèm vào chương trình hoặc tệp tin chủ và không phụ thuộc vào hành động của người dùng để lây nhiễm – nó tự động khai thác lỗ hổng bảo mật trong hệ thống để cài mã độc và sao chép bản thân. Nói cách khác, chỉ cần một máy tính bị

nhiễm ban đầu, sâu có thể tự động phát tán rộng khắp các thiết bị khác thông qua các kết nối mạng nội bộ hoặc Internet, gây ảnh hưởng quy mô lớn.

Sâu máy tính lây lan chủ yếu bằng cách khai thác lỗ hổng trong giao thức mạng, hệ điều hành hoặc phần mềm dịch vụ. Chúng có thể “luồn lách” vào qua các cổng mở (open port) hoặc kênh chia sẻ tập tin, sau đó tự động quét và tấn công các máy khác trên cùng mạng. Ngoài ra, một số loại sâu phổ biến còn lan truyền qua tập tin đính kèm email và thiết bị USB. Ví dụ, nhiều sâu sẽ gửi email đại trà kèm theo tập tin đính kèm chứa mã độc đến toàn bộ địa chỉ liên lạc của nạn nhân – khi người dùng mở tệp này, sâu sẽ cài vào máy và tự gửi tiếp cho các liên hệ khác. Tương tự, khi cắm USB chứa sâu vào máy tính, mã độc có thể tự động sao chép từ USB vào hệ thống và rồi lây lan tiếp cho các thiết bị USB khác được kết nối sau đó.

Vì tự nhân bản rất nhanh và quét mạng liên tục, sâu máy tính có thể chiếm dụng băng thông và làm tắc nghẽn hệ thống mạng, khiến máy tính và dịch vụ trực tuyến bị chậm hoặc ngưng trệ. Ở mức độ lớn, sâu có thể gây ra DDoS (tấn công từ chối dịch vụ) bằng cách gửi hàng loạt lưu lượng giả mạo, làm sập máy chủ hoặc toàn bộ hạ tầng mạng. Thực tế cho thấy các cuộc tấn công sâu nguy hiểm từng gây thiệt hại kinh tế rất lớn: chẳng hạn sâu ILOVEYOU (2000) lây lan qua email đã nhiễm hàng chục triệu máy và gây thiệt hại hàng tỷ đô la, còn sâu WannaCry (2017) đã mã hóa dữ liệu và lan truyền khắp thế giới qua lỗ hổng SMB của Windows, ước tính gây thiệt hại hàng trăm triệu đến hàng tỷ đô la.

Thêm vào đó, nhiều loại sâu còn hoạt động như kênh trung gian để tải về hoặc cài đặt các phần mềm độc hại khác. Ví dụ, sau khi xâm nhập, sâu có thể tự động cài thêm spyware, ransomware hoặc mở cửa hậu (backdoor) cho kẻ tấn công, để chiếm quyền điều khiển máy tính nạn nhân sau này. Tóm lại, sâu máy tính (worm) là mối đe dọa nghiêm trọng vì khả năng lây lan tự động rộng khắp và gây ra gián đoạn hệ

thống cùng tổn thất kinh tế đáng kể. Để ngăn chặn sâu, cần cập nhật bản vá bảo mật kịp thời và sử dụng giải pháp an ninh mạng phù hợp.

d. Rootkit

Rootkit là một dạng phần mềm độc hại được thiết kế với mục đích che giấu sự hiện diện của chính nó và các chương trình nguy hiểm khác trong hệ thống máy tính. Không giống như virus có khả năng tự nhân bản và lây lan, rootkit thường không hoạt động độc lập mà đi kèm với các mã độc khác nhằm nâng cao hiệu quả ẩn giấu và kiểm soát.

Thuật ngữ "rootkit" xuất hiện lần đầu trong báo cáo bảo mật CA-1994-01 được công bố bởi CERT Coordination Center (CERT/CC) vào tháng 2 năm 1994, liên quan đến các cuộc tấn công giám sát mạng. Ban đầu, rootkit chủ yếu được dùng để duy trì quyền truy cập trái phép trong hệ thống Unix bằng cách thay thế các tệp hệ thống, đặc biệt là trên tài khoản "root" – tài khoản có quyền cao nhất trong hệ thống.

Về bản chất, rootkit không trực tiếp gây hại mà đóng vai trò như một công cụ che đậy. Chúng có thể can thiệp sâu vào hệ điều hành, ở cấp độ kernel (nhân hệ điều hành), thậm chí có thể ẩn mình ở firmware hoặc phần cứng, khiến cho việc phát hiện trở nên vô cùng khó khăn. Nhờ khả năng này, các mã độc như spyware, keylogger, trojan hoặc ransomware khi hoạt động dưới sự bảo vệ của rootkit sẽ gần như "vô hình" trước phần mềm diệt virus thông thường.

Theo thời gian, công nghệ rootkit ngày càng tinh vi hơn. Những phiên bản hiện đại có thể thao túng bộ nạp hệ điều hành (bootloader), bảng điều khiển hệ thống (kernel patching), hoặc thậm chí can thiệp vào trình điều khiển thiết bị (driver hooking). Chính vì vậy, rootkit được xem là một trong những mối đe dọa nguy hiểm nhất đối với an ninh mạng hiện đại.

Việc phát hiện và loại bỏ rootkit đòi hỏi các công cụ chuyên biệt và kiến thức chuyên sâu, chẳng hạn như phân tích bộ nhớ (memory forensics), so sánh checksum hệ thống hoặc sử dụng hệ điều hành sạch để quét ngoại tuyến.

e. Botnet

Bot (viết tắt của "robot") là một dạng phần mềm độc hại được cài đặt âm thầm trên máy tính nạn nhân, biến thiết bị này trở thành một "zombie" – tức là có thể bị điều khiển từ xa mà người dùng không hề hay biết. Khi nhiều máy tính bị nhiễm cùng một loại bot, chúng sẽ hình thành một mạng lưới gọi là Botnet – một tập hợp các thiết bị bị kiểm soát từ xa bởi kẻ tấn công thông qua máy chủ điều khiển trung tâm (C&C – Command and Control).

Tương tự như phần mềm Backdoor, các bot cho phép tin tặc từ xa ra lệnh thực hiện nhiều hành vi độc hại trên máy tính nạn nhân. Các mệnh lệnh có thể được truyền qua nhiều kênh khác nhau như Internet Relay Chat (IRC), hệ thống ngang hàng (P2P), hoặc thậm chí các giao thức ẩn danh và mã hóa nhằm che giấu hoạt động liên lạc.

Khi đã nắm trong tay một mạng Botnet đủ lớn, kẻ điều khiển có thể sử dụng nó cho nhiều mục đích như:

- Tấn công từ chối dịch vụ phân tán (DDoS) để làm tê liệt website hoặc hạ tầng mạng.
- Gửi thư rác (spam) quy mô lớn.
- Phát tán phần mềm độc hại khác, như ransomware hoặc spyware.
- Đánh cắp dữ liệu cá nhân từ máy nạn nhân.

Botnet ngày nay không chỉ là công cụ của các cá nhân tội phạm mạng, mà còn được sử dụng trong các cuộc chiến tranh mạng giữa các quốc gia hoặc tổ chức, với mục tiêu làm gián đoạn hạ tầng số hoặc đánh cắp thông tin mật.

f. Ransomware

Ransomware là một dạng phần mềm độc hại cực kỳ nguy hiểm, hoạt động bằng cách mã hóa dữ liệu trên hệ thống máy tính của nạn nhân và yêu cầu một khoản tiền chuộc để cung cấp khóa giải mã. Trong vài năm trở lại đây, ransomware đã trở thành một trong những mối đe dọa an ninh mạng nghiêm trọng nhất, với tần suất và mức độ gây hại ngày càng gia tăng. Loại mã độc này từng khiến nhiều cơ quan, doanh nghiệp – từ công ty tư nhân, bệnh viện, cơ quan thực thi pháp luật cho đến toàn bộ thành phố – rơi vào tình trạng tê liệt hoạt động.

Phần lớn ransomware thuộc họ Trojan, tức là không có khả năng tự lây lan mà phải phụ thuộc vào kỹ thuật truyền nhiễm như email lừa đảo (phishing), các liên kết giả mạo hoặc phần mềm tải về từ nguồn không tin cậy. Sau khi được thực thi, ransomware thường tìm kiếm và mã hóa các tệp dữ liệu quan trọng trong thời gian ngắn, đôi khi chỉ vài phút. Một số biến thể tinh vi hơn lại sử dụng chiến lược trì hoãn, quan sát hành vi người dùng trong vài giờ để lựa chọn thời điểm tấn công hiệu quả, đồng thời xác định giá trị dữ liệu để đưa ra mức tiền chuộc hợp lý – có thể kèm theo hành vi mã hóa cả bản sao lưu nhằm gây sức ép tối đa.

Việc ngăn chặn ransomware có thể thực hiện thông qua các biện pháp an ninh truyền thống như phần mềm chống virus, tường lửa, đào tạo người dùng và bảo vệ email. Tuy nhiên, khi ransomware đã được kích hoạt, việc phục hồi dữ liệu trở nên rất khó khăn nếu không có bản sao lưu an toàn và tách biệt khỏi hệ thống chính (offline). Theo thống kê, khoảng 25% nạn nhân chấp nhận trả tiền chuộc, nhưng 30%

trong số đó vẫn không thể phục hồi dữ liệu, cho thấy không có gì đảm bảo kẻ tấn công sẽ giữ lời.

Đặc biệt, ransomware hiện đại không còn đơn thuần chỉ khóa thiết bị (locker) hay đưa ra cảnh báo giả (scareware) như các phiên bản trước đây. Thay vào đó, chúng sử dụng mã hóa tiên tiến, gần như không thể phá vỡ nếu không có khóa giải mã, làm gia tăng đáng kể tính nghiêm trọng và độ nguy hiểm của loại phần mềm độc hại này.

1.1.4.2 Xác định loại mã độc bằng dịch vụ virustotal

Trong khuôn khổ bài báo cáo này, mã băm MD5 của các mẫu mã độc được sử dụng để tra cứu thông tin trên VirusTotal nhằm xác định tên và loại mã độc tương ứng. VirusTotal là một dịch vụ web nổi tiếng, chuyên dùng để phân tích và nhận diện phần mềm độc hại. Nền tảng này cho phép người dùng tải lên tập tin hoặc cung cấp mã băm (MD5, SHA1, SHA256) để quét và kiểm tra bằng hàng chục công cụ diệt virus uy tín từ nhiều hãng bảo mật khác nhau.

VirusTotal không chỉ giúp phát hiện nhanh chóng các loại mã độc như virus, worm, trojan, spyware, ransomware,... mà còn hỗ trợ phân tích hành vi và cung cấp thông tin chi tiết về tệp tin, bao gồm: tên mẫu mã độc được các hãng bảo mật nhận diện, thời điểm phát hiện, mức độ nguy hiểm, liên kết đến các tài nguyên liên quan, v.v. Nhờ đó, VirusTotal trở thành công cụ hỗ trợ đắc lực trong quá trình nghiên cứu, phân tích và đánh giá phần mềm độc hại.

1.2. Kỹ thuật phân tích mã độc

Phân tích mã độc là một bước quan trọng trong lĩnh vực an toàn thông tin, giúp hiểu rõ cách thức hoạt động, hành vi và mục tiêu của phần mềm độc hại. Có hai kỹ thuật chính được sử dụng phổ biến là phân tích tĩnh và phân tích động. Dưới đây là mô tả chi tiết về hai kỹ thuật:

1.2.1 Phân tích động

Phân tích động (Dynamic Analysis) là kỹ thuật đánh giá hành vi của mã độc bằng cách cho phép nó thực thi trong một môi trường được kiểm soát, thường là máy ảo hoặc sandbox, từ đó quan sát và ghi lại các hành vi thay đổi hệ thống trong quá trình hoạt động.

Các công cụ phổ biến được sử dụng trong phân tích động bao gồm:

- SysAnalyzer: Theo dõi trạng thái hệ thống và các thay đổi liên quan đến tiến trình, cổng mạng, driver, thư viện, registry, API được gọi, và lưu lượng DNS.
- Process Explorer, ProcMon: Giám sát tiến trình và hoạt động hệ thống theo thời gian thực.
- RegShot: So sánh các thay đổi trong registry trước và sau khi mã độc thực thi.
- IDA Pro, OllyDbg, Immunity Debugger: Hỗ trợ dịch ngược mã độc và phân tích mã máy.

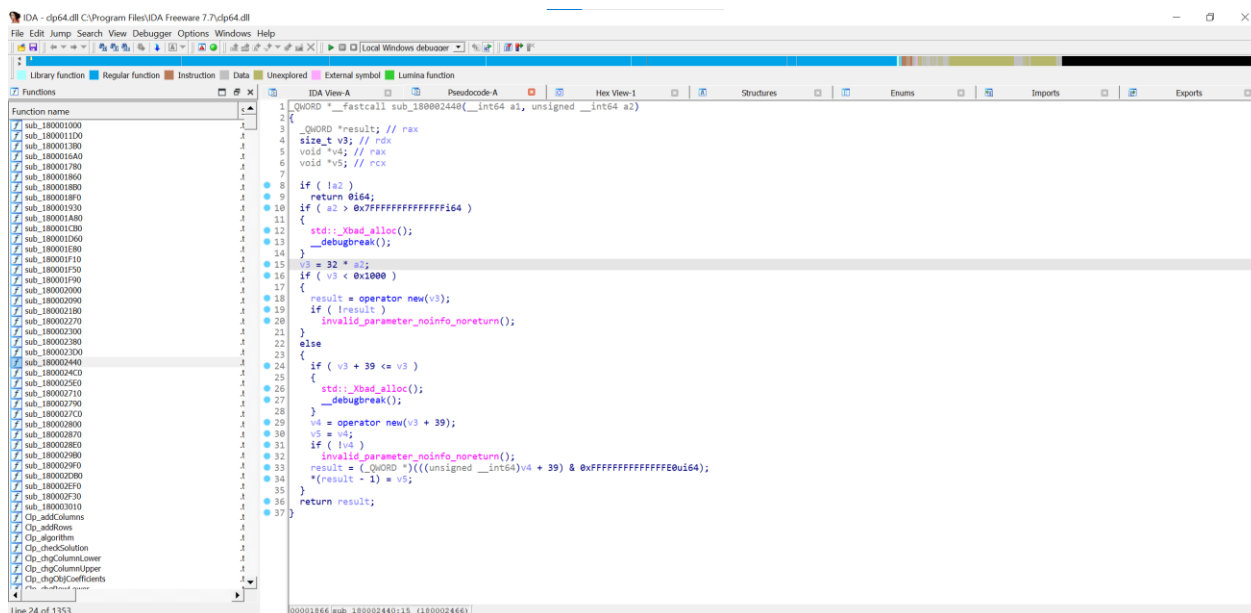
Phân tích động cho phép các chuyên gia an ninh phát hiện ra mục đích và hành vi thực sự của mã độc – điều đôi khi khó đạt được nếu chỉ dùng phân tích tĩnh. Tuy nhiên, kỹ thuật này đòi hỏi thời gian và kiến thức sâu về kiến trúc hệ thống, vì các mẫu mã độc hiện đại thường được đóng gói (packed) hoặc mã hóa (encrypted) nhằm tránh bị phát hiện. Đóng gói là chương trình chỉnh sửa các tệp tin của chương trình khác để nén nội dung của chúng. Khi chương trình đóng gói nén, mã hóa, hoặc chỉnh sửa một chương trình thực thi, thì chương trình đó và chương trình gốc về bên ngoài sẽ khác nhau. Trong các trường hợp như vậy, mã độc cần được giải nén hoặc giải mã trước khi tiến hành phân tích.

Ví dụ, SysAnalyzer là công cụ nổi bật trong phân tích động, hỗ trợ theo dõi hoạt động hệ thống và các tiến trình đáng ngờ, nhờ đó cung cấp cái nhìn toàn diện về hành vi của mã độc trong quá trình chạy.

1.2.2 Phân tích tĩnh

Phân tích tĩnh (Static Analysis) là kỹ thuật phân tích mã độc mà không cần thực thi chương trình, thông qua việc kiểm tra mã nguồn, mã máy hoặc mã đã được dịch ngược để xác định hành vi và cấu trúc bên trong của mã độc.

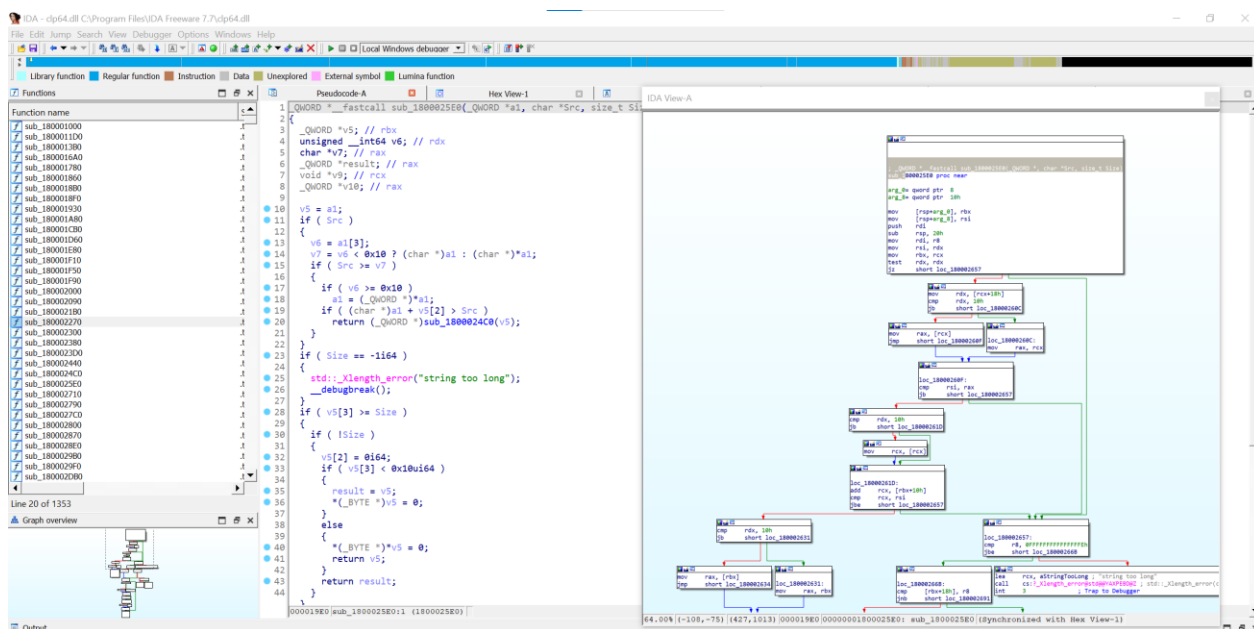
Một trong những công cụ nổi bật và phổ biến nhất cho phân tích tĩnh là IDA Pro. Đây là một trình disassembler (trình dịch ngược) mạnh mẽ, cho phép người dùng quan sát mã nguồn dưới dạng mã hợp ngữ (assembly). Một trong những tính năng đặc biệt của IDA Pro là khả năng chuyển đổi mã hợp ngữ sang mã giả C (pseudocode), giúp các nhà phân tích dễ hình dung hơn logic hoạt động của chương trình.



Hình 1.4: Chức năng Pseudocode-A tạo mã giả C của IDA pro

Ngoài ra, IDA Pro còn cung cấp chế độ hiển thị đồ họa (graph overview), giúp người phân tích dễ dàng hình dung các nhánh thực thi và cấu trúc hàm của chương trình một cách trực quan.

Người dùng cũng có thể nhanh chóng chuyển tới các hàm cụ thể bằng cách nhấp đúp vào tên hàm trong cửa sổ danh sách hàm – tính năng này hỗ trợ rất tốt trong việc điều hướng giữa các thành phần mã độc phức tạp.



Hình 1.5: Chương trình được phân tích bởi IDA pro

Tuy nhiên, việc phân tích tĩnh đòi hỏi kiến thức chuyên sâu và kinh nghiệm từ phía nhà phân tích. Mã giả C được sinh ra bởi phần mềm thường không rõ ràng như mã gốc, khiến việc đọc hiểu trở nên khó khăn. Thêm vào đó, nhiều mã độc hiện đại sử dụng các kỹ thuật chống phân tích tĩnh như:

- Obfuscation (mã rối)
- Dummy code (mã giả)
- Anti-disassembly (chống dịch ngược)

Những kỹ thuật này khiến cho quá trình phân tích trở nên phức tạp hơn và đòi hỏi chuyên gia phải kết hợp thêm kỹ thuật phân tích động để xác định chính xác hành vi của mã độc trong ngữ cảnh thực thi.

1.3 Kết luận chương 1

Chương 1 đã cung cấp cái nhìn tổng quan về các kỹ thuật phân tích mã độc hiện nay, bao gồm phân tích tĩnh và phân tích động. Tuy các phương pháp hiện tại có độ chính xác cao, nhưng vẫn tồn tại nhiều hạn chế, đặc biệt là về thời gian phân tích và khả năng tiếp cận nhanh chóng các mẫu mã độc mới. Từ đó, bài toán được đặt ra là làm sao để tối ưu hóa quy trình phân tích, rút ngắn thời gian xử lý mà vẫn đảm bảo độ chính xác cao trong việc phát hiện mã độc. Đây cũng chính là mục tiêu hướng đến trong các chương tiếp theo của bài báo cáo. Ngoài ra, chương này cũng đã trình bày một số loại mã độc phổ biến như virus, worm, trojan, rootkit, ransomware, botnet,... nhằm xây dựng nền tảng kiến thức cơ bản để hỗ trợ cho quá trình phát hiện và phân tích các mẫu mã độc sau này.

CHƯƠNG 2. PHÁT HIỆN MÃ ĐỘC DỰA TRÊN METADATA CỦA TẬP TIN

Chương này trình bày chi tiết về cấu trúc tệp thực thi Win32, còn được gọi là cấu trúc tệp PE (Portable Executable), vốn là định dạng tệp phổ biến trong hệ điều hành Windows. Cấu trúc PE đóng vai trò quan trọng trong việc phân tích và phát hiện mã độc, vì nó chứa nhiều thông tin dạng meta-data có thể sử dụng làm đặc trưng đầu vào cho các thuật toán học máy. Chương sẽ mô tả cấu trúc tổng quát của tệp PE, từ đó lựa chọn các trường thông tin phù hợp để trích xuất dữ liệu huấn luyện cho mô hình. Việc khai thác meta-data từ tệp thực thi cho phép ta phát hiện gián tiếp hành vi mã độc mà không cần thực thi mẫu thực tế, giúp giảm thiểu rủi ro và thời gian phân tích. Cuối cùng, chương sẽ giới thiệu các kỹ thuật học máy được sử dụng để đánh giá – cụ thể là thuật toán Random Forest, Decision Tree, Naive Bayes và Artificial Neural Network

2.1 Khái niệm về meta-data

2.1.1 Định nghĩa

Siêu dữ liệu (metadata) dùng để mô tả tài nguyên thông tin. Thuật ngữ “meta” xuất xứ là một từ Hy Lạp dùng để chỉ một cái gì đó có bản chất cơ bản hơn hoặc cao hơn. Vì vậy metadata là dữ liệu về dữ liệu.

Theo tiến sĩ Warwick Cathro (Thư viện Quốc gia Úc) thì “siêu dữ liệu là những thành phần mô tả tài nguyên thông tin hoặc hỗ trợ thông tin truy cập đến tài nguyên thông tin”. Cụ thể trong tài liệu thì siêu dữ liệu được xác định là “dữ liệu mô tả các thuộc tính của đối tượng thông tin và trao cho các thuộc tính này ý nghĩa, khung cảnh và tổ chức. Siêu dữ liệu còn có thể được định nghĩa là dữ liệu có cấu trúc về dữ liệu”.

Theo Gail Hodge siêu dữ liệu là “thông tin có cấu trúc mà nó mô tả, giải thích, định vị, hoặc làm cho nguồn tin trở nên dễ tìm kiếm, sử dụng và quản lý hơn. Siêu dữ liệu được hiểu là dữ liệu về dữ liệu hoặc thông tin về thông tin”

Nói tóm lại thì siêu dữ liệu là thông tin mô tả tài nguyên thông tin.

Mục đích đầu tiên và yêu cầu cốt lõi nhất của siêu dữ liệu (metadata) là góp phần mô tả và tìm lại các tài liệu điện tử trên mạng Internet. Sự phát triển mạnh mẽ của Internet đã tạo ra sự bùng nổ của các loại dữ liệu đa dạng ở dạng số, văn bản, âm thanh, hình ảnh, tài liệu đa phương tiện. Những tài liệu này có thể truy cập được trên mạng Internet song việc tìm kiếm chúng một cách hiệu quả và khoa học như với các hệ thống thông tin trực tuyến là hết sức khó khăn. Để góp phần tăng cường chất lượng tìm kiếm các tài liệu số trên mạng Internet, người ta đã đưa ra giải pháp sử dụng siêu dữ liệu.

Thực ra trong hoạt động thông tin – thư viện truyền thống, từ lâu đã có những khái niệm liên quan đến siêu dữ liệu. Các bản thư mục chứa các dữ liệu mô tả đối tượng như cho sách, cho tạp chí thì chúng cũng được coi như là một dạng siêu dữ liệu. Với việc tự động hóa công tác biên mục, phiếu thư mục được thay thế bằng biểu ghi thư mục. Như vậy thành phần siêu dữ liệu còn có thể được trình bày trong biểu ghi, vì vậy biểu ghi này được coi là biểu ghi siêu dữ liệu (metadata record) của đối tượng được cơ sở dữ liệu quản lý. Với tài nguyên truyền thống trên giấy, thông tin mô tả được bố trí nằm ngoài đối tượng mà nó mô tả (Ví dụ, trên phiếu thư mục của mục lục thư viện, trong biểu ghi của CSDL). Nhờ những yếu tố mô tả như vậy, người ta có thể xác định và tìm kiếm lại được tài liệu một cách chính xác theo một vài yếu tố.

Ngày nay, nguồn tài liệu điện tử phát triển nhanh chóng và sự phân tán trên mạng nhiều đến mức không thể xử lý được một cách thủ công như đã và đang áp dụng đối với tài liệu xuất bản trên giấy. Để xử lý được hết tài liệu điện tử phân tán, người ta phải áp dụng các phương pháp tự động – sử dụng các chương trình đặc biệt (được gọi theo nhiều cách khác nhau như: robots, crawlers, spiders,...). Do tài liệu điện tử được tạo ra, thông thường không tuân thủ những quy định xuất bản truyền thống, không có những quy tắc nhất định giúp cho phép nhận dạng tự động được các yếu tố mô tả thông thường như tác giả, địa chỉ về xuất bản, thông tin về khối lượng... nên cần thiết phải có những quy định thống nhất để các chương trình tự động nhận dạng và xử lý chúng theo các yêu cầu nghiệp vụ. Những quy định như vậy được gọi là những quy định về siêu dữ liệu. Có thể thấy hiện nay, do nhiều chương trình máy tính chỉ định chỉ số dựa vào một số thành phần hạn chế như nhan đề hoặc toàn văn nên không hỗ trợ những tìm kiếm đặc thù (ví dụ theo tác giả, theo chủ đề, theo lĩnh vực...). Vì thế để tạo điều kiện cho các chương trình có thể định chỉ số tự động theo một số yếu tố xác định, người ta phải đưa thêm vào tài liệu điện tử những thuộc tính bổ sung để tăng cường mô tả tài nguyên thông tin. Các công cụ định chỉ số tự động sẽ được lập trình để nhận dạng các thuộc tính này và định chỉ số chúng, từ đó hỗ trợ tìm kiếm những thuộc tính đặc thù.

Như vậy một bản ghi metadata bao gồm một tập hợp những thuộc tính hoặc tập hợp những phần tử cần thiết để mô tả các tài nguyên thông tin theo yêu cầu nghiệp vụ. Thông thường trong hoạt động nghiệp vụ thông tin – thư viện bao gồm các yếu tố như: Nhan đề tài liệu, tác giả, thông tin về xuất bản, nơi/vị trí lưu giữ, kiểu/dạng tài liệu....

2.1.2 Cách sử dụng siêu dữ liệu

Mối liên hệ giữa siêu dữ liệu và tài nguyên thông tin mà nó mô tả có thể được thể hiện ở một trong hai cách sau:

- Các phần tử metadata được chứa trong một biểu ghi tách biệt bên ngoài đối tượng mô tả.
- Các phần tử metadata có thể được nhúng (gắn) vào bên trong tài nguyên mà nó mô tả.

Trước đây với tài liệu truyền thống, các mô tả dữ liệu nằm ngoài đối tượng mô tả (được đưa vào phiếu thư viện hoặc biểu ghi CSDL), như vậy siêu dữ liệu được lưu trữ một cách tách biệt bên ngoài đối tượng mô tả.

Với tài liệu điện tử, siêu dữ liệu của chúng được nhúng (gắn) trong bản thân tài nguyên hoặc liên kết với tài nguyên mà nó mô tả như trong trường hợp các thẻ meta của tài liệu HTML hoặc các tiêu đề TEI (Text Encoding Initiative) trong tài liệu điện tử.

Trong thực tế có nhiều chuẩn mô tả biên mục mang tính chất metadata khá thông dụng đang được áp dụng như: MARC21/UNIMARC, ISO-2709, Dublin Core Metadata... các dữ liệu metadata này thường được gắn vào phần đầu cho mỗi tài liệu điện tử được đưa vào máy chủ hoặc trên mạng internet nhằm hỗ trợ các công cụ tìm kiếm lọc ra các thông tin metadata để tổ chức thành các kho dữ liệu mà không cần dùng đến hệ quản trị cơ sở dữ liệu truyền thống. Thực tế thì ngay bản thân ngôn ngữ XML tự nó đã hỗ trợ việc hình thành một cơ sở dữ liệu toàn văn, phi cấu trúc và rất thuận lợi cho việc tìm kiếm và trao đổi thông tin.

2.1.3 Mô tả dữ liệu

Để thống nhất phương thức mô tả tài liệu theo một khuôn mẫu thống nhất, người ta đưa ra những sơ đồ siêu dữ liệu. Việc này tương tự như cộng đồng thư viện thống nhất sử dụng khổ mẫu Marc21 hoặc UNIMARC cho biên mục đọc máy để đảm bảo tính thống nhất trong tạo lập CSDL. Vì thế hiện nay đã xuất hiện nhiều sơ đồ siêu dữ liệu khác nhau với quy định ngữ nghĩa riêng của mình.

Sơ đồ dữ liệu (Metadata scheme): Là tập hợp những yếu tố siêu dữ liệu được thiết kế cho mô tả một dạng tài nguyên thông tin cụ thể. Như vậy siêu dữ liệu là sơ đồ hình thức được xác định để mô tả tài nguyên thông tin cho đối tượng số hoặc không số. Thí dụ tập hợp yếu tố siêu dữ liệu Dublin Core có sơ đồ bao gồm 15 yếu tố để mô tả tài nguyên thông tin. Các nhà khoa học nhân văn đã xây dựng sơ đồ TEI (Text Encoding Initiative) để thống nhất tập hợp các yếu tố để mô tả tài liệu số đưa lên mạng hoặc đưa vào CSDL số. Các nhà lưu trữ tài liệu điện tử thì khuyến nghị sử dụng sơ đồ EAD (Encoded Archival Description) do họ đề xuất để thống nhất khổ mẫu cho lưu trữ tài liệu điện tử.

Ngữ nghĩa (Semantics) : Định nghĩa các yếu tố hoặc ý nghĩa được gán cho các yếu tố siêu dữ liệu thì được gọi là ngữ nghĩa của sơ đồ. Mỗi sơ đồ siêu dữ liệu có ngữ nghĩa và cú pháp được quy định riêng. Ví dụ bộ yếu tố siêu dữ liệu Dublin Core chỉ có 15 yếu tố, trong đó yếu tố “Creator” – dùng để xác định là tác giả của tài liệu, hoặc yếu tố “Title” – được hiểu là nhan đề của tài liệu.

Nội dung (Content): Giá trị (dữ liệu) của từng yếu tố được gọi là nội dung. Đó chính là giá trị của mỗi yếu tố siêu dữ liệu. Nhờ các sơ đồ dữ liệu, các chương trình xử lý tự động sẽ nhận biết đoạn dữ liệu nào sẽ thuộc thành phần nào, chẳng hạn đoạn

dữ liệu này được nhận biết là nhân đề, đoạn dữ liệu kia được nhận biết là tác giả của tài liệu.

Việc tạo lập siêu dữ liệu hiện nay phần lớn vẫn được thực hiện theo phương pháp thủ công, trong khi các công cụ tự động hỗ trợ còn nhiều hạn chế, đặc biệt là thiếu tính năng kiểm tra tính hợp lệ và đồng nhất dữ liệu. Bên cạnh đó, việc bảo trì siêu dữ liệu cũng gặp nhiều khó khăn. Những thay đổi trong các quy trình nghiệp vụ như thu thập, sao lưu, bảo quản hay phục vụ khai thác tài liệu đều kéo theo các thay đổi trong bản thân dữ liệu. Khi đó, siêu dữ liệu tương ứng cũng cần được cập nhật một cách chính xác và kịp thời. Nếu không có sự đồng bộ này, khả năng truy xuất hoặc xác thực dữ liệu trong tương lai có thể bị ảnh hưởng nghiêm trọng, thậm chí không thể đọc được dữ liệu đã lưu trữ.

Một vấn đề quan trọng khác là sự thiếu hụt các tiêu chuẩn thống nhất để trao đổi siêu dữ liệu giữa các hệ thống khác nhau. Hiện nay, có hai hướng phát triển lớn trong lĩnh vực chuẩn hóa siêu dữ liệu. Một được khởi xướng bởi Microsoft và nhận được sự hỗ trợ từ Liên minh Meta Data Coalition – một tập hợp gồm hơn 50 nhà sản xuất và người dùng. Hướng còn lại đến từ sự hợp tác giữa IBM và Oracle. Cả hai đề xuất này đều dựa trên nền tảng XML (Extensible Markup Language) nhằm tạo ra khả năng trao đổi và tương tác dữ liệu linh hoạt giữa các hệ thống. Ngoài ra, một số nhà phát triển phần mềm độc lập cũng đang tích cực mở rộng ứng dụng siêu dữ liệu trên nền XML, đồng thời hỗ trợ cả hai hệ chuẩn trên.

Tuy nhiên, theo đánh giá của giới chuyên môn, toàn bộ quá trình chuẩn hóa siêu dữ liệu hiện vẫn đang trong giai đoạn hoàn thiện và chưa đạt được sự đồng thuận toàn diện.

Trong lĩnh vực lưu trữ, các vấn đề liên quan đến vòng đời của tài liệu điện tử – từ khâu tạo lập, sao lưu định kỳ, bảo quản lâu dài, đến khai thác và sử dụng – đang ngày càng trở nên cấp thiết. Đặc biệt, cần có sự đầu tư nghiên cứu bài bản nhằm bảo đảm tài liệu điện tử có thể đọc được trong tương lai, bất chấp thay đổi về phần cứng hay phần mềm. Bên cạnh đó, tính xác thực và giá trị pháp lý của tài liệu điện tử cũng cần được bảo đảm trong quá trình khai thác. Việc xây dựng và ban hành các tiêu chuẩn dữ liệu nói chung, đặc biệt là tiêu chuẩn về đặc tả dữ liệu và siêu dữ liệu trong quản lý tài liệu điện tử, cần có sự tham gia chủ động của các cơ quan quản lý nhà nước về văn thư, lưu trữ.

Trong lĩnh vực an toàn thông tin, đặc biệt là trong bài toán phát hiện mã độc, siêu dữ liệu (meta-data) của tệp tin đóng vai trò là đặc trưng đầu vào quan trọng. Các siêu dữ liệu này được trích xuất từ các trường trong cấu trúc tệp tin thực thi (PE – Portable Executable), bao gồm sơ đồ cấu trúc vật lý, thuộc tính, và các thông tin mô tả kỹ thuật khác. Xét trên phương diện kỹ thuật, đây là những dữ liệu quan trọng giúp các nhà nghiên cứu an ninh mạng xác định liệu một tệp tin có khả năng thuộc về một họ mã độc cụ thể hay không. Các đặc trưng này sẽ được trình bày chi tiết trong Chương 3: *Triển khai và đánh giá*.

2.2 Cấu trúc tệp tin thực thi trên hệ điều hành Windows

2.2.1 Khái niệm PE File

Portable Executable (PE) là định dạng tệp được Microsoft thiết kế dành riêng cho các hệ điều hành Windows sử dụng kiến trúc Win32 hoặc Win64. Đây là định dạng được áp dụng phổ biến cho hầu hết các tệp có khả năng thực thi trên Windows như: *.EXE (ứng dụng thực thi), *.DLL (thư viện liên kết động), *.SYS (trình điều khiển ở chế độ nhân), *.CPL (các tiện ích trong Control Panel), *.OCX (điều khiển

ActiveX), cũng như các ứng dụng được phát triển trên nền tảng .NET. Việc sử dụng định dạng PE cho cả ứng dụng và thư viện cho phép hệ điều hành quản lý, tải và thực thi chúng một cách linh hoạt, hiệu quả.

Tuy nhiên, vẫn tồn tại một số ngoại lệ không tuân theo định dạng PE, bao gồm các tệp 16-bit sử dụng định dạng NE (New Executable), các tệp VxD trên Windows 9x, và các tệp *.COM vốn được sử dụng phổ biến trong môi trường MS-DOS. Đặc biệt, các trình điều khiển kernel-mode trên các hệ điều hành Windows NT và các phiên bản kế tiếp cũng sử dụng định dạng PE để hệ thống có thể dễ dàng quản lý và thực hiện các thao tác nạp vào bộ nhớ khi khởi động.

Việc nghiên cứu định dạng PE giúp hiểu rõ hơn về cách hệ điều hành Windows tổ chức và thực thi các chương trình. Cấu trúc PE chứa nhiều trường thông tin quan trọng, như các header, bảng nhập xuất (import/export table), section chứa mã và dữ liệu, và nhiều thành phần khác. Những thông tin này cực kỳ hữu ích trong các lĩnh vực như phân tích mã độc, kỹ thuật đảo ngược (reverse engineering) và phát triển công cụ bảo mật.

2.2.2 Cấu trúc cơ bản của một PE File

Một tệp tin PE được chia thành hai phần chính là Header và Section. Phần Header đóng vai trò định nghĩa định dạng của tệp và cung cấp các thông tin offset để truy xuất đến các section trong phần thân tệp. Header bao gồm các thành phần như DOS Header, PE Header (gồm Signature, COFF Header và Optional Header), cùng với bảng mô tả các section (Section Table).

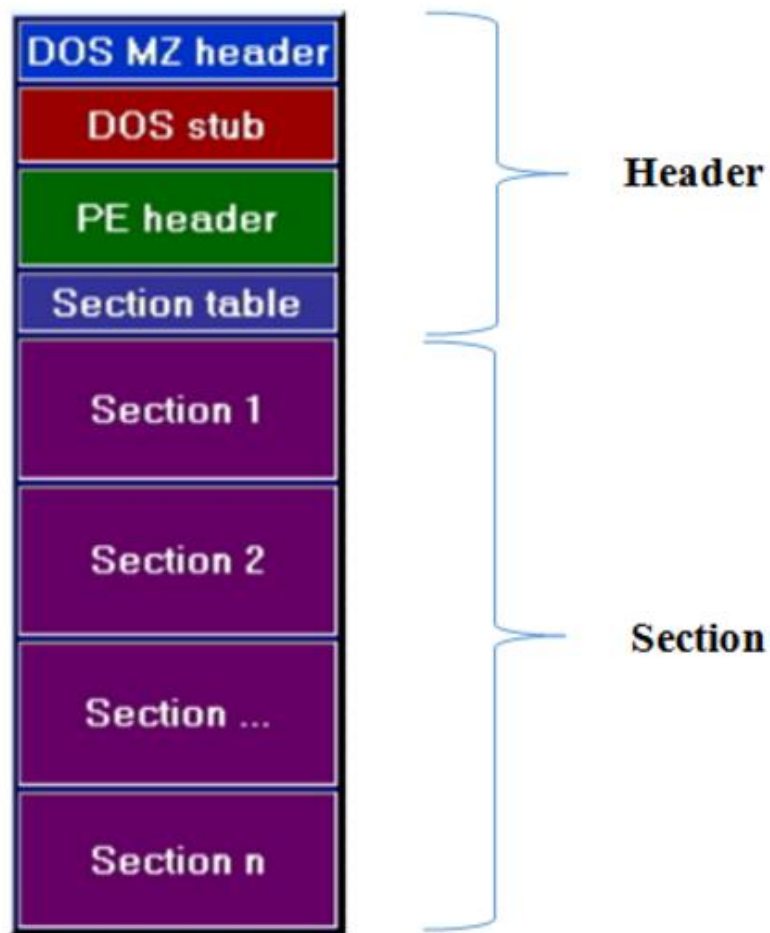
Phần thứ hai – Section – là nơi lưu trữ nội dung thực thi và dữ liệu của chương trình. Ở mức tối thiểu, một tệp PE phải có ít nhất hai section: một section dùng cho đoạn mã (code), và một section cho dữ liệu (data). Tuy nhiên, trong các chương trình

ứng dụng thực tế, số lượng section có thể nhiều hơn để phục vụ các chức năng chuyên biệt.

Trên nền tảng Windows NT, có tổng cộng 9 section tiêu chuẩn thường được sử dụng. Những section này bao gồm:

- .text: chứa mã thực thi
- .data: chứa dữ liệu đã khởi tạo
- .bss: lưu dữ liệu chưa khởi tạo
- .rdata: chứa dữ liệu chỉ đọc (ví dụ: hằng số)
- .rsrc: lưu trữ tài nguyên (như biểu tượng, hộp thoại,...)
- .edata: chứa bảng xuất (export table)
- .idata: lưu bảng nhập (import table)
- .pdata: chứa thông tin xử lý ngoại lệ
- .debug: lưu thông tin phục vụ quá trình gỡ lỗi

Tùy theo trình biên dịch và tính chất của ứng dụng, không phải chương trình nào cũng sử dụng đầy đủ các section này. Một số ứng dụng nhỏ chỉ cần một vài section, trong khi các phần mềm phức tạp hơn có thể mở rộng thêm nhiều section khác để đáp ứng yêu cầu xử lý chuyên biệt.



Hình 2.1: Cấu trúc tập tin thực thi PE

2.2.3 DOS MZ Header

Tất cả các tệp PE (Portable Executable) đều bắt đầu bằng một DOS MZ Header, một thành phần đơn giản nhưng quan trọng, chiếm 64 byte đầu tiên của tệp. Thành phần này được đặt tên theo hai ký tự đầu tiên là “MZ”, viết tắt tên của Mark Zbikowski – một trong những kỹ sư của Microsoft đã thiết kế định dạng này. Mục đích chính của DOS Header là để đảm bảo khả năng tương thích ngược với hệ điều hành DOS. Khi một chương trình PE được chạy trong môi trường DOS, hệ điều hành sẽ nhận diện đây là một tệp thực thi hợp lệ thông qua MZ Header và thực thi phần

mã nằm trong DOS Stub – thường hiển thị thông báo như “This program cannot be run in DOS mode”. Điều này đảm bảo rằng tệp PE không bị hệ thống cũ từ chối xử lý và cho phép cung cấp phản hồi rõ ràng cho người dùng.

```
struct _IMAGE_DOS_HEADER {  
0x00 WORD e_magic;  
0x02 WORD e_cblp;  
0x04 WORD e_cp;  
0x06 WORD e_crlc;  
0x08 WORD e_cparhdr;  
0x0a WORD e_minalloc;  
0x0c WORD e_maxalloc;  
0x0e WORD e_ss;  
0x10 WORD e_sp;  
0x12 WORD e_csum;  
0x14 WORD e_ip;  
0x16 WORD e_cs;  
0x18 WORD e_lfarlc;  
0x1a WORD e_ovno;  
0x1c WORD e_res[4];  
0x24 WORD e_oemid;  
0x26 WORD e_oeminfo;  
0x28 WORD e_res2[10];  
0x3c DWORD e_lfanew;  
};
```

Hình 2.2: Cấu trúc Image_Dos_Header

Trong đó chúng ta cần quan tâm tới hai trường:

- **e_magic**: Chữ ký của PE file, giá trị: 4Dh, 5Ah (Ký tự “MZ”, tên của người sáng lập MS-DOS: Mark Zbikowsky). Giá trị này đánh dấu một DOS Header hợp lệ và được phép thực thi tiếp.
- **e_lfanew**: là một DWORD nằm ở cuối cùng của DOS Header, là trường chứa offset của PE Header so với vị trí đầu file.

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	
00000000h:	4D	5A	50	00	02	00	00	00	04	00	0F	00	FF	FF	00	00	; MZ.....yy.. DOS header signature in e_magic
00000010h:	B8	00	00	00	00	00	00	00	40	00	1A	00	00	00	00	00	;@.....
00000020h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000030h:	00	00	00	00	00	00	00	00	00	00	00	00	00	01	00	00	; e_lfanew contains offset of PE header (NB bytes in reverse order: 00 00 01 00)
00000040h:	BA	10	00	0E	1F	B4	09	CD	21	B8	01	4C	CD	21	90	90	; ^.....'í!..Lí![]
00000050h:	54	68	69	73	20	70	72	6F	67	72	61	6D	20	6D	75	73	; This program mus
00000060h:	74	20	62	65	20	72	75	6E	20	75	6E	64	65	72	20	57	; t be run under W
00000070h:	69	6E	33	32	0D	0A	24	37	00	00	00	00	00	00	00	00	; in32..\$7..... DOS stub
00000080h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000090h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000a0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000b0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000c0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000d0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000e0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000f0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000100h:	50	45	00	00	4C	01	08	00	19	5E	42	2A	00	00	00	00	; PE.....^B*... PE signature at start of PE header
00000110h:	00	00	00	00	E0	00	8E	81	08	01	02	19	00	A0	02	00	;à.2[].....

Hình 2.3: Giá trị của lfanew trên hex dump

2.2.4 DOS STUB

DOS Stub là một chương trình nhỏ được nhúng ngay sau phần DOS MZ Header trong cấu trúc của một tệp PE. Chức năng chính của DOS Stub là đảm bảo khả năng tương thích với các hệ thống cũ, đặc biệt là môi trường Windows 16-bit hoặc hệ điều hành MS-DOS. Trong trường hợp người dùng cố gắng chạy một tệp PE trong môi trường không hỗ trợ định dạng này, DOS Stub sẽ được thực thi thay vì phần mã chính của chương trình. Thông thường, nó chỉ hiển thị một thông báo đơn giản như: *"This program cannot be run in DOS mode"*. Mặc dù phần này không còn có vai trò trong quá trình thực thi trên các hệ điều hành Windows hiện đại, nhưng nó vẫn là một phần không thể thiếu trong cấu trúc chuẩn của PE để duy trì tính tương thích ngược.

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	
00000000	4D	5A	90	00	03	00	00	00	04	00	00	00	FF	FF	00	00	MZ.....ÿÿ..
00000010	B8	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	,.....@.....
00000020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000030	00	00	00	00	00	00	00	00	00	00	00	00	80	00	00	00	€.....
00000040	0E	1F	BA	0E	00	B4	09	CD	21	B8	01	4C	CD	21	54	68	..°..'.Í!..LÍ!Th
00000050	69	73	20	70	72	6F	67	72	61	6D	20	63	61	6E	6E	6F	is program cannot
00000060	74	20	62	65	20	72	75	6E	20	69	6E	20	44	4F	53	20	t be run in DOS
00000070	6D	6F	64	65	2E	0D	0D	0A	24	00	00	00	00	00	00	00	mode....\$......
00000080	50	45	00	00	64	86	11	00	4C	62	5C	59	00	34	0F	00	PE..d+..Lb\Y.4..
00000090	D0	54	00	00	F0	00	27	00	0B	02	02	18	00	0A	07	00	DT..đ.'.....
000000A0	00	4E	0A	00	00	18	00	00	00	15	00	00	00	10	00	00	.N.....
000000B0	00	00	40	00	00	00	00	00	00	10	00	00	00	02	00	00	..@.....
000000C0	04	00	00	00	00	00	00	00	05	00	02	00	00	00	00	00	

Hình 2.4: Giá trị của DOS STUB trên hexdump

2.2.5 PE Header

PE Header là một thành phần quan trọng trong cấu trúc của tệp PE (Portable Executable), thực chất được định nghĩa bởi cấu trúc `IMAGE_NT_HEADERS`. Đây là nơi chứa các thông tin cần thiết để hệ điều hành thực hiện việc nạp (load) tệp thực thi vào bộ nhớ, thông qua bộ nạp (loader) của Windows. Cấu trúc `IMAGE_NT_HEADERS` bao gồm ba phần chính: chữ ký (Signature), cấu trúc `IMAGE_FILE_HEADER` mô tả các thuộc tính chung của file, và `IMAGE_OPTIONAL_HEADER` chứa thông tin chi tiết như địa chỉ entry point, căn chỉnh bộ nhớ, và vị trí các bảng dữ liệu. Tất cả những phần này đều được định nghĩa rõ ràng trong tệp tin `windows.inc` của Windows SDK hoặc các công cụ lập trình tương đương, tạo nên nền tảng cho quá trình xử lý và phân tích tệp PE.

```
struct _IMAGE_NT_HEADERS {
    0x00  DWORD Signature;
    0x04  _IMAGE_FILE_HEADER FileHeader;
    0x18  _IMAGE_OPTIONAL_HEADER OptionalHeader;
};
```

Hình 2.5: Cấu trúc `Image_NT_Header`

Signature: là phần đầu tiên của PE Header, có độ dài 4 byte (một DWORD), dùng để đánh dấu sự bắt đầu của định dạng PE. Giá trị mặc định của trường này là 50h 45h 00h 00h, tương ứng với chuỗi ký tự ASCII “PE\0\0”. Đây là chữ ký đặc trưng cho tất cả các tệp tin sử dụng định dạng Portable Executable, cho phép hệ điều hành và các công cụ phân tích nhận diện chính xác đây là một tệp PE hợp lệ.

FileHeader: là phần nằm ngay sau trường Signature trong cấu trúc PE Header, có độ dài 20 byte. Phần này chứa các thông tin quan trọng phản ánh sơ đồ bố trí vật lý cũng như các đặc điểm cơ bản của tệp tin thực thi.

```
struct _IMAGE_FILE_HEADER {  
    0x00 WORD Machine;  
    0x02 WORD NumberOfSections;  
    0x04 DWORD TimeDateStamp;  
    0x08 DWORD PointerToSymbolTable;  
    0x0c DWORD NumberOfSymbols;  
    0x10 WORD SizeOfOptionalHeader;  
    0x12 WORD Characteristics;  
};
```

Hình 2.6: Cấu trúc Image_File_Header

- **Machine:** Xác định loại bộ vi xử lý đã được dùng để biên dịch tệp tin.
- **NumberOfSections:** Số lượng các section (phân đoạn) có trong tệp PE.
- **TimeDateStamp:** Thời điểm tệp tin được tạo ra, được lưu theo định dạng thời gian Unix (Unix Time).
- **SizeOfOptionalHeader:** Kích thước (tính bằng byte) của phần OptionalHeader trong tệp PE.
- **Characteristics:** Các đặc tính định dạng của tệp tin, ví dụ như tệp thực thi (.exe), thư viện liên kết động (.dll), v.v.

OPTIONAL_HEADER: của tệp PE, kích thước tiêu chuẩn là 224 bytes, trong đó 128 bytes cuối cùng được dành để lưu thông tin về Data Directory. Cấu trúc đầy đủ của phần này được định nghĩa dưới dạng IMAGE_OPTIONAL_HEADER32, bao gồm nhiều trường quan trọng mô tả sơ đồ logic để hệ điều hành biết cách nạp và thực thi chương trình.

```
struct _IMAGE_OPTIONAL_HEADER {
0x00 WORD Magic;
0x02 BYTE MajorLinkerVersion;
0x03 BYTE MinorLinkerVersion;
0x04 DWORD SizeOfCode;
0x08 DWORD SizeOfInitializedData;
0x0c DWORD SizeOfUninitializedData;
0x10 DWORD AddressOfEntryPoint;
0x14 DWORD BaseOfCode;
0x18 DWORD BaseOfData;
0x1c DWORD ImageBase;
0x20 DWORD SectionAlignment;
0x24 DWORD FileAlignment;
0x28 WORD MajorOperatingSystemVersion;
0x2a WORD MinorOperatingSystemVersion;
0x2c WORD MajorImageVersion;
0x2e WORD MinorImageVersion;
0x30 WORD MajorSubsystemVersion;
0x32 WORD MinorSubsystemVersion;
0x34 DWORD Win32VersionValue;
0x38 DWORD SizeOfImage;
0x3c DWORD SizeOfHeaders;
0x40 DWORD CheckSum;
0x44 WORD Subsystem;
0x46 WORD DllCharacteristics;
0x48 DWORD SizeOfStackReserve;
0x4c DWORD SizeOfStackCommit;
0x50 DWORD SizeOfHeapReserve;
0x54 DWORD SizeOfHeapCommit;
0x58 DWORD LoaderFlags;
0x5c DWORD NumberOfRvaAndSizes;
0x60 _IMAGE_DATA_DIRECTORY DataDirectory[16];
};
```

Hình 2.7: Cấu trúc Image_Optional_Header32

Trong đó, các thành phần được mô tả như sau:

- **Magic** (2 bytes): xác định là file 32 bit (0B 01) hay 64 bit (0B 20)

- **AddressOfEntryPoint** (4bytes): chứa địa chỉ ảo tương đối (RVA) của câu lệnh đầu tiên sẽ được thực thi khi chương trình PE loader sẵn sàng để chạy PE File (.text). Nếu muốn chương trình bắt đầu từ một địa chỉ khác (để thực thi câu lệnh với mục đích khác) thì cần thay đổi địa chỉ này về địa chỉ tương đối của câu lệnh muốn thực thi.
- **ImageBase**: địa chỉ nạp được ưu tiên cho PE File.
- **Section Alignment**: Phần liên kết của các Section trong bộ nhớ. Tức là một section luôn luôn được bắt đầu bằng bội số của sectionAlignment. Ví dụ: sectionAlignment là 1000h, section đầu tiên bắt đầu ở vị trí 401000h và kích thước là 10h, section tiếp theo sẽ bắt đầu tại địa chỉ 402000h.
- **File Alignment**: Phần liên kết của các Section trong File. Tương tự như SectionAlignment nhưng áp dụng với file.
- **SizeOfImage**: Toàn bộ kích thước của Pe image trong bộ nhớ, là tổng của tất cả các headers và sections được liên kết tới Section Alignment
- **SizeOfHeaders**: Kích thước của tất cả các headers + section table = kích thước file trừ đi tổng kích thước của các section trong file.

2.2.6 Data Directory

Trong cấu trúc tệp PE, phần Data Directory là một mảng gồm 16 cấu trúc IMAGE_DATA_DIRECTORY, mỗi cấu trúc này chứa thông tin về các thành phần dữ liệu quan trọng có trong tệp PE. Mảng này chiếm 128 byte cuối cùng của Optional Header và là một trong những thành phần cuối cùng của cấu trúc IMAGE_NT_HEADERS. Mỗi mục trong mảng Data Directory chứa hai thông tin quan trọng: VirtualAddress và Size. Cụ thể:

- **VirtualAddress**: Đây là địa chỉ ảo tương đối (RVA) của cấu trúc dữ liệu trong bộ nhớ khi tệp PE được nạp vào.

- **Size:** Kích thước của cấu trúc dữ liệu trong tệp PE.

```

IMAGE_DATA_DIRECTORY STRUCT
    VirtualAddress    DWORD    ?
    isize             DWORD    ?
IMAGE_DATA_DIRECTORY ENDS

```

Hình 2.8: Cấu trúc Image_Data_Directory

Các cấu trúc trong Data Directory đóng vai trò quan trọng trong việc tổ chức và quản lý thông tin cần thiết cho quá trình thực thi của tệp PE. Dưới đây là danh sách các mục có trong Data Directory:

- **Export Table:** Mảng này chứa thông tin về các hàm và biến được xuất ra bên ngoài tệp PE. Đây là một phần quan trọng đối với các tệp DLL, giúp các chương trình khác có thể sử dụng các hàm bên trong tệp này.
- **Import Table:** Chứa thông tin về các hàm và biến mà tệp PE cần nhập từ các tệp DLL khác. Điều này cho phép tệp PE liên kết động với các thư viện khi chạy.
- **Resource Table:** Bao gồm tất cả các tài nguyên được tích hợp vào trong tệp PE như hình ảnh, chuỗi văn bản, biểu tượng, v.v. Đây là thành phần giúp tệp PE dễ dàng truy cập và sử dụng tài nguyên khi cần thiết.
- **Exception Table:** Cung cấp thông tin về việc xử lý ngoại lệ, đặc biệt hữu ích đối với các ngôn ngữ lập trình như C++ trong việc quản lý các ngoại lệ trong quá trình thực thi.
- **Certificate Table:** Lưu trữ chữ ký số (Authenticode) của tệp PE, đảm bảo tính xác thực và an toàn của tệp trong các hệ thống bảo mật.

- **Base Relocation Table:** Chứa thông tin giúp di chuyển các địa chỉ trong tệp PE khi tệp không được nạp vào đúng **ImageBase** ban đầu. Điều này rất quan trọng đối với các tệp có kích thước lớn hoặc cần được nạp vào vị trí bộ nhớ khác nhau.
- **Debug Directory:** Đây là phần chứa thông tin phục vụ quá trình gỡ lỗi tệp PE, cung cấp các thông tin quan trọng cho các công cụ gỡ lỗi.
- **Architecture-specific Data:** Dữ liệu đặc biệt cho các kiến trúc phần cứng nhất định. Mặc dù không thường xuyên được sử dụng, mục này giúp định dạng PE có thể tương thích với các nền tảng phần cứng khác nhau.
- **Global Pointer Table:** Thành phần này chỉ sử dụng trong các hệ thống IA-64, giúp quản lý các dữ liệu toàn cục trong các chương trình.
- **TLS Table:** Cung cấp thông tin về **Thread Local Storage (TLS)**, nơi lưu trữ các biến có phạm vi tồn tại chỉ trong một luồng nhất định, giúp tối ưu hóa việc xử lý đa luồng.
- **Load Configuration Table:** Chứa cấu hình cần thiết cho quá trình tải tệp PE vào bộ nhớ. Đây là phần quan trọng giúp tối ưu hóa quá trình khởi tạo và thực thi tệp PE.
- **Bound Import Table:** Tăng tốc quá trình liên kết động bằng cách xác định trước các DLL mà tệp PE cần sử dụng.
- **Import Address Table (IAT):** Chứa địa chỉ thực tế của các hàm nhập từ các tệp DLL, giúp tệp PE có thể thực thi đúng các hàm nhập này trong quá trình thực thi.
- **Delay Import Descriptor:** Dùng để trì hoãn việc nhập các DLL cho đến khi thực sự cần thiết, giúp tiết kiệm tài nguyên hệ thống và giảm độ trễ trong quá trình khởi động tệp PE.

- **CLR Runtime Header:** Cấu trúc này xuất hiện trong các tệp PE của ứng dụng .NET, cung cấp các thông tin cần thiết để ứng dụng .NET có thể thực thi trên nền tảng Windows.
- **Reserved:** Mục này hiện tại chưa được sử dụng nhưng có thể được dành cho các mục khác trong các phiên bản tương lai của định dạng PE.

	RVA	Size	...	L	H
ExportTable:	00000000	00000000	...	L	H
ImportTable:	0002D000	0000181E	...	L	H
Resource:	00034000	00008E00	...	L	H
Exception:	00000000	00000000		L	H
Security:	00000000	00000000			H
Relocation:	00031000	00002B04	...	L	H
Debug:	00000000	00000000	...	L	H
Copyright:	00000000	00000000	...	L	H
Globalptr:	00000000	00000000			
TlsTable:	00030000	00000018	...	L	H
LoadConfig:	00000000	00000000		L	H
BoundImport:	00000000	00000000	...	L	H
IAT:	00000000	00000000			H
DelayImport:	00000000	00000000		L	H
COM:	00000000	00000000	...	L	H
Reserved:	00000000	00000000			H

Hình 2.9: Cấu trúc dữ liệu được định nghĩa bởi Data Directory

2.2.7 Section Table

Section Table là phần tiếp theo ngay sau PE Header trong tệp tin PE. Nó bao gồm một mảng các cấu trúc Image_Section_Header, mỗi phần tử trong mảng này chứa thông tin về một section trong tệp PE, chẳng hạn như các thuộc tính của section và địa chỉ tương đối ảo (virtual offset).

Số lượng section trong tệp PE được xác định bởi trường NumberOfSections, nằm trong phần thứ hai của cấu trúc FileHeader (khoảng 6 byte tính từ đầu PE Header). Ví dụ, nếu tệp PE có 8 sections, thì Section Table sẽ có 8 bản sao của cấu trúc Image_Section_Header.

Mỗi cấu trúc Image_Section_Header có kích thước 40 byte và sẽ không có thêm "padding" giữa các cấu trúc này (Padding ở đây có nghĩa là không thêm các byte có giá trị 00h). Cấu trúc của mỗi section header được định nghĩa như trong hình dưới đây:

```
IMAGE_SECTION_HEADER STRUCT
    Name1                BYTE                IMAGE_SIZEOF_SHORT_NAME dup(?)
    union Misc
        PhysicalAddress   DWORD            ?
        VirtualSize       DWORD            ?
    ends
    VirtualAddress        DWORD            ?
    SizeOfRawData         DWORD            ?
    PointerToRawData      DWORD            ?
    PointerToRelocations  DWORD            ?
    PointerToLinenumbers  DWORD            ?
    NumberOfRelocations   WORD             ?
    NumberOfLinenumbers   WORD             ?
    Characteristics       DWORD            ?
IMAGE_SECTION_HEADER ENDS

IMAGE_SIZEOF_SHORT_NAME equ 8
```

Hình 2.10: Cấu trúc Image_Section_Header

Ý nghĩa của các trường trong cấu trúc Image_Section_Header:

- **Name1:** Đây là một nhãn cho section, nhưng không phải là một chuỗi ASCII và có thể để trống. Điều này có nghĩa là nó không cần kết thúc bằng ký tự null (0).
- **VirtualSize:** Kích thước của section trong bộ nhớ, tính bằng byte. Kích thước này có thể nhỏ hơn kích thước của section trên đĩa, và đây là kích thước mà Windows Loader sẽ dùng để xác định vị trí của section trong bộ nhớ.
- **VirtualAddress:** Địa chỉ ảo tương đối của section. Trình PE Loader sẽ sử dụng giá trị này khi ánh xạ section vào bộ nhớ. Ví dụ, nếu giá trị trong trường này là 1000h và tệp tin PE được nạp vào địa chỉ 40000h, thì section này sẽ được nạp vào địa chỉ 401000h.
- **SizeOfRawData:** Kích thước của section trên đĩa, được làm tròn lên bội số tiếp theo của giá trị này. Đây là cách Windows quản lý bộ nhớ theo cơ chế phân trang.
- **PointerToRawData:** Đây là offset từ vị trí bắt đầu của tệp tin đến phần dữ liệu của section. Nếu giá trị trong trường này là 0, điều đó có nghĩa là section data không tồn tại trong tệp tin và không cần phải nạp vào bộ nhớ. Trình PE Loader sẽ sử dụng giá trị này để xác định vị trí của phần dữ liệu trong section trong tệp tin.
- **Characteristics:** Trường này chứa các cờ đặc biệt, ví dụ như chỉ ra rằng section này có thể chứa mã thực thi, dữ liệu đã khởi tạo, dữ liệu chưa khởi tạo, hoặc nó có thể được ghi hoặc đọc.

Ngoài các trường quan trọng đã nêu trên, tệp tin PE còn có nhiều cấu trúc khác như **Import Directory**, **Export Directory**, và các cấu trúc khác liên quan đến các

tính năng bổ sung. Tuy nhiên, trong phạm vi bài báo cáo này, chúng em không sử dụng các cấu trúc đó để huấn luyện và sẽ không đề cập chi tiết về chúng. Các trường khác cũng được trích xuất và liệt kê trong phần triển khai.

2.3 Các mô hình học máy

2.3.1. Mô hình cây quyết định Decision Tree

Decision Tree (Cây quyết định) là một mô hình học máy được sử dụng cho cả hai bài toán phân loại và hồi quy. Mô hình hoạt động bằng cách phân chia không gian đặc trưng (predictor space) thành các vùng nhỏ dựa trên giá trị của các biến đầu vào, sau đó đưa ra dự đoán dựa trên trung bình (đối với hồi quy) hoặc nhãn xuất hiện nhiều nhất (đối với phân loại) của các quan sát trong mỗi vùng.

Để xây dựng một cây quyết định, sử dụng phép chia nhị phân đệ quy theo phương pháp top-down và greedy, bắt đầu từ đỉnh và tại mỗi bước thuật toán sẽ chọn biến và điểm chia tốt nhất, quá trình sẽ được lặp lại cho tới khi đạt được một tiêu chí dừng. Sau khi các vùng $R_1, \dots, R_j$ được tạo ra, chúng ta đưa ra dự đoán cho dữ liệu đầu vào tùy thuộc vào vùng mà dữ liệu đó rơi vào và loại phân loại được sử dụng (hồi quy hoặc tuyến tính).

Đối với cây hồi quy, thuật toán sẽ chọn biến và điểm chia tốt nhất để giảm sai số RSS (Residual sum of Squares) lớn nhất theo công thức:

$$\sum_{i: x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2$$

Trong đó $R_1(j, s)$ và $R_2(j, s)$ là hai vùng chia tại bước đó theo biến X_j và điểm cắt s .

Đối với cây phân loại, thuật toán sẽ lựa chọn biến và điểm chia tốt nhất dựa trên chỉ số Gini:

$$G = \sum_{k=1}^K \hat{p}_{mk} (1 - \hat{p}_{mk})$$

Hoặc là sử dụng entropy:

$$D = - \sum_{k=1}^K \hat{p}_{mk} \log(\hat{p}_{mk})$$

Trong đó, $\hat{p}_{mk}$ đại diện cho tỷ lệ quan sát huấn luyện trong vùng thứ m của nhãn thứ k.

Ưu điểm của cây quyết định là dễ hiểu và dễ diễn giải một cách trực quan, phù hợp với dữ liệu có biến định tính, tuy nhiên có nhược điểm là dễ bị overfitting và độ chính xác thường thấp hơn các phương pháp khác.

2.3.2 Mô hình Rừng ngẫu nhiên Random Forest

2.3.2.1 Phương pháp học máy tổ hợp Ensemble method

Phương pháp học máy tổ hợp (Ensemble method) là một phương pháp huấn luyện nhiều mô hình học để giải quyết một vấn đề. Trái với cách tiếp cận học máy thông thường là chỉ thực hiện một mô hình từ một dữ liệu huấn luyện. Học máy tổ hợp thực hiện một tập các mô hình học máy và kết hợp chúng và đưa ra dự đoán. Cấu trúc của một học máy tổ hợp được tạo thành từ nhiều mô hình học cơ sở (có thể là cây quyết định, hồi quy tuyến tính hoặc các thuật toán học máy cơ sở khác). Để phân loại, các phương pháp học máy tổ hợp có thể sử dụng duy nhất một thuật toán làm mô hình học cơ sở hoặc sử dụng nhiều thuật toán để tạo ra các mô hình học cơ

sở. Một cách phân loại khác nữa đó chính là dựa trên cách thức của phương pháp học máy tổ hợp, bao gồm hai loại là boosting và bagging. Tuy nhiên, do chỉ đề cập tới Random Forest, là một phương pháp học máy mở rộng của bagging, nên nhóm chúng em sẽ không đề cập tới boosting.

2.3.2.2 Bagging – Bootstrap and Aggregating

Bagging viết tắt của Bootstrap (lấy lại mẫu có thay thế) và AGGREGATING (tổng hợp). Giống với tên gọi, thuật toán này gồm hai thành phần chính đó là lấy lại mẫu với thay thế và tổng hợp. Đầu tiên, sử dụng phương pháp lấy lại mẫu có thay thế để tạo N tập con dữ liệu để huấn luyện các mô hình học cơ sở. Sau đó, nó sẽ tổng hợp hay đưa ra kết quả bằng phương pháp bình chọn (voting) cho phân loại và phương pháp trung bình (averaging) cho hồi quy dựa trên mỗi kết quả đầu ra của các mô hình học cơ sở. Mục đích của bagging là giảm phương sai (variance) bằng cách trung bình hóa kết quả của nhiều mô hình có sự đa dạng.

2.3.2.3 Random Forest

Mặc dù phương pháp bagging giúp giảm phương sai và cải thiện lỗi tổng quát hóa (generalization error). Tuy nhiên, nếu các cây được huấn luyện quá giống nhau (do nếu có một số đặc trưng đầu vào quá mạnh, tất cả các cây đều có xu hướng chọn các biến này cho phép chia đầu tiên), việc trung bình hóa sẽ không mang lại hiệu quả rõ rệt. Rừng ngẫu nhiên (Random Forest) ra đời để khắc phục vấn đề này bằng cách giảm sự tương quan giữa các cây. Sự khác biệt giữa RF so với bagging là việc đưa sự ngẫu nhiên vào huấn luyện. Trong việc xác định cây quyết định thành phần, thay vì xét toàn bộ p biến đầu vào, nó chỉ xét ngẫu nhiên một tập con gồm m biến, sau đó thực hiện quá trình chọn điểm chia với m biến đó như cây quyết định. Và rừng ngẫu nhiên được tạo nên bởi n các cây quyết định thành phần với các tập con m đặc trưng khác nhau được lấy ngẫu nhiên. Ưu điểm của Rừng ngẫu nhiên là

hiệu suất tốt hơn so với bagging, đặc biệt là dữ liệu có nhiều biến đầu vào, chống overfitting, ngay cả khi số lượng cây lớn. Tuy nhiên, nhược điểm của Rừng ngẫu nhiên là thiếu khả năng giải thích (do kết hợp rất nhiều cây quyết định với nhau) và tốn tài nguyên tính toán khi phải xử lý rất nhiều cây.

2.3.3. Mô hình Naive Bayes

Naive Bayes là một phương pháp phân loại dựa trên định lý Bayes với giả định rằng các đặc trưng là độc lập có điều kiện với nhau khi biết nhãn lớp. Giả định này dù không thường đúng trên thực tế vẫn cho kết quả tốt cho nhiều bài toán phân loại, đặc biệt là khi tập dữ liệu có kích thước nhỏ hoặc khi cần tốc độ huấn luyện và suy luận nhanh. Mô hình Naive Bayes phân loại dựa trên công thức:

$$c = \underset{c \in \{1, \dots, C\}}{\operatorname{argmax}} p(c) \prod_{i=1}^d p(x_i | c)$$

Trong đó:

- $p(c)$: xác suất tiên nghiệm của nhãn c
- $p(x_i | c)$: xác suất có điều kiện của thuộc tính x_i với điều kiện là nhãn c

Khi số chiều dữ liệu lớn hoặc các xác suất rất nhỏ, dẫn đến biểu thức ở vế phải là một số rất nhỏ, khi tính toán có thể xảy ra sai số, ta thường sử dụng log để tính toán:

$$c = \underset{c \in \{1, \dots, C\}}{\operatorname{argmax}} \left(\log(p(c)) + \sum_{i=1}^d \log(p(x_i | c)) \right)$$

Việc tính toán $p(x_i | c)$ phụ thuộc vào loại dữ liệu. Có ba loại phân bố xác suất phổ biến là Gaussian Naive Bayes đối với thành phần là các biến liên tục, Multinomial Naive Bayes dùng trong bài toán phân loại văn bản và Bernoulli Naive Bayes đối với thành phần là giá trị nhị phân. Trong bài báo cáo này chúng ta sử dụng Gaussian Naive Bayes.

CHƯƠNG 3. TRIỂN KHAI THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

3.1 Triển khai giải pháp

3.1.1 Giới thiệu giải pháp

Việc phát hiện và phân loại phần mềm độc hại hiệu quả có thể được thực hiện dựa trên việc phân tích PE (Portable Executable) Header của tệp thực thi (.EXE, .DLL,...) kết hợp với các kỹ thuật học máy khác nhau.

Phương pháp học máy đóng một vai trò quan trọng trong việc phát hiện mã độc tự động. Nhiệm vụ thách thức lớn nhất là chọn một bộ đặc trưng phù hợp từ một tập dữ liệu lớn để có thể xây dựng mô hình phân loại trong thời gian ngắn hơn với độ chính xác cao nhất có thể. Mục đích của báo cáo này trước hết là thử nghiệm đánh giá, xem xét tổng thể các phương pháp phân loại, phát hiện mã độc và thứ hai là phát triển một hệ thống tự động để phát hiện mã độc dựa trên PE Header của file thực thi với độ chính xác cao và thời gian thực hiện nhanh chóng.

Để thực hiện, đầu tiên thu thập một tập một lượng lớn file thực thi .EXE hoặc .DLL, gồm cả file mã độc và file sạch. Sau đó thực hiện phân tích các thông tin từ PE Header. Sau đó thực hiện khảo sát thống kê và lựa chọn thông tin đặc trưng quan trọng từ PE Header. Tiếp theo là áp dụng các kỹ thuật học máy khác nhau để phân loại và phát hiện mã độc. Sau cùng là tiến hành thử nghiệm, đánh giá độ chính xác và so sánh kết quả nghiên cứu so với các kết quả hiện hành.

3.1.2 Các bước triển khai

Thu thập dữ liệu:

- Thu thập các mẫu mã độc dạng PE file từ nhiều nguồn: Virushare, Malware DB. Malshare, ... Ta thu được 5400 file malware từ các nguồn trên.
- Thu thập tệp tin thông thường: lấy các file sạch dạng PE File trong thư mục Windows, tải về từ trang chủ của Microsoft, FileHippo. Ta thu được 1060 file Benign từ các file thư mục trên máy tính cá nhân.

Làm sạch dữ liệu (Data Cleaning)

- Loại bỏ các tệp lỗi.
- Loại bỏ các mẫu trên hệ điều hành Linux, MacOS, Android,...
- Loại bỏ các tệp không phải định dạng PE.
- Loại bỏ một số tệp mà độc nguy hiểm có khả năng gây nguy hiểm cho máy khi giải nén.
- Lọc các tệp trùng nhau.
- Thu được 1.047 mẫu mã độc và 1.033 mẫu file sạch dùng để tạo bộ dữ liệu sử dụng để xây dựng mô hình.

Lựa chọn đặc trưng

- **IMAGE_DOS_HEADER** gồm 19 đặc trưng:

"e_magic", "e_cblp", "e_cp", "e_crlc", "e_cparhdr", "e_minalloc", "e_maxmallocl", "e_ss", "e_sp", "e_csum", "e_ip", "e_cs", "e_lfarlc", "e_ovno", "e_res", "e_oemid", "e_oeminfo", "e_res2", "e_lfanew".

- **FILE_HEADER** gồm 7 đặc trưng:

"Machine", "NumberOfSection", "CreationYear", "PointerToSymbolTable", "NumberOfSymbols", "SizeOfOptionalHeader", "Characteristics".

- **OPTIONAL_HEADER** gồm 29 đặc trưng:

"Magic", "MajorLinkerVersion", "MinorLinkerVersion", "SizeOfCode", "SizeOfInitializedData", "SizeOfUninitializedData", "AddressOfEntryPoint", "BaseOfCode", "BaseOfData", "ImageBase", "SectionAlignment", "FileAlignment", "MajorOperatingSystemVersion", "MinorOperatingSystemVersion", "MajorImageVersion", "MinorImageVersion", "MajorSubsystemVersion", "MinorSubsystemVersion", "SizeOfImage", "SizeOfHeaders", "Checksum", "Subsystem", "DllCharacteristics", "SizeOfStackReserve", "SizeOfStackCommit", "SizeOfHeapReserve", "SizeOfHeapCommit", "LoaderFlags", "NumberOfRvaAndSizes".

Xử lý dữ liệu (Data Preprocessing)

- “e_res” và “e_res2” là các trường dự trữ (reserved) trong PE header, không mang ý nghĩa cụ thể nên thường bị loại bỏ khỏi đặc trưng huấn luyện để tránh nhiễu và overfitting.
- Fill in missing values: Cấu trúc tệp PE là cố định và các thành phần optional nếu không có đều có giá trị 0 trong bước trích rút.
- Outliers và noisy data: Các file là các mẫu mã độc và file sạch có thể thực thi được do đó không có noisy data. Các outlier có thể do tính chất của từng ứng dụng và từng mã độc. Không thực hiện việc loại bỏ outlier và noisy data, do bất cứ việc thay thế giá trị nào đều không thể hiện được bản chất thực sự của tệp PE.

Gán nhãn dữ liệu

- Mã độc có giá trị là 1
- Tập tin sạch giá trị là 0

Filename	Address	Offset	Base	Offset	Character	Checksum	Creation	Y	DllCharac	FileAlign	ImageBase	LoaderFile	Machine	Magic	MajorIma	MajorLink	MajorOpe	MajorSub	MinorIma	MinorLink	MinorOpe	MinorSub	NumberOf	NumberOf	NumberOf	PointerTo	SectionAll
2025-06-1	13199	4096	32768	271	0	2018	34112	512	4194304	0	332	267	6	6	4	4	0	0	0	0	0	16	5	0	0	4096	
2025-06-1	82606	4096	188416	258	562830	2023	33088	512	4194304	0	332	267	0	14	6	6	0	30	0	0	0	16	5	0	0	4096	
2025-06-1	147772	4096	0	34	0	2024	33120	512	5.4E+09	0	34404	523	0	14	6	6	0	35	0	0	0	16	6	0	0	4096	
2025-06-1	26572	4096	32768	258	3269738	2013	33088	512	4194304	0	332	267	6	11	6	5	3	0	3	1	16	5	0	0	4096		
2025-06-1	6095536	4096	0	34	7.1E+07	2024	49504	512	5.4E+09	0	34404	523	0	14	6	6	0	41	0	0	0	16	10	0	0	4096	
2025-06-1	113918	8192	0	258	199190	2076	34144	512	4194304	0	332	267	0	8	4	4	0	0	0	0	0	16	3	0	0	8192	
2025-06-1	6068432	4096	0	34	6.9E+07	2025	49504	512	5.4E+09	0	34404	523	0	14	6	6	0	42	0	0	0	16	10	0	0	4096	
2025-06-1	2.9E+07	4096	0	34	8.6E+07	2025	33120	512	5.4E+09	0	34404	523	0	14	6	6	0	42	0	0	0	16	8	0	0	4096	
2025-06-1	26096	4096	0	34	0	2025	33120	512	5.4E+09	0	34404	523	0	14	6	6	0	0	0	0	0	16	10	0	0	4096	
2025-06-1	6068432	4096	0	34	7.6E+07	2025	49504	512	5.4E+09	0	34404	523	0	14	6	6	0	42	0	0	0	16	10	0	0	4096	
2025-06-1	2.9E+07	4096	0	34	8.6E+07	2025	33120	512	5.4E+09	0	34404	523	0	14	6	6	0	42	0	0	0	16	8	0	0	4096	
2025-06-1	689084	4096	692224	258	0	2024	33088	512	4194304	0	332	267	0	2	6	6	0	25	1	1	16	11	0	0	4096		
2025-06-1	2.9E+07	4096	0	34	8.6E+07	2025	33120	512	5.4E+09	0	34404	523	0	14	6	6	0	42	0	0	0	16	8	0	0	4096	
2025-06-2	5806292	4096	0	34	0	2025	33120	512	5.4E+09	0	34404	523	0	14	6	6	0	42	0	0	0	16	6	0	0	4096	
2025-06-2	5148	4096	0	34	67600	2021	33120	512	5.4E+09	0	34404	523	0	14	6	6	0	16	0	0	0	16	6	0	0	4096	
2025-06-2	5088	4096	0	550	95715	2024	352	512	5.4E+09	0	34404	523	0	2	4	5	0	43	0	2	16	11	58	44032	4096		
2025-06-2	5806292	4096	0	34	0	2025	33120	512	5.4E+09	0	34404	523	0	14	6	6	0	42	0	0	0	16	6	0	0	4096	
2025-06-2	82144	4096	0	34	0	2024	49504	512	5.4E+09	0	34404	523	0	14	6	6	0	39	0	0	0	16	7	0	0	4096	
2025-06-2	3800894	8192	0	290	0	2066	34144	512	4194304	0	332	267	0	48	4	4	0	0	0	0	0	16	3	0	0	8192	
malware,	5344	4096	0	559	0	2019	0	512	4194304	0	34404	523	0	2	4	5	0	30	0	2	16	9	0	0	4096		
malware,	4128736	3592192	0	34	0	2019	33120	512	5.4E+09	0	34404	523	0	14	6	6	0	22	0	0	0	16	3	0	0	4096	
malware,	4124400	3588096	0	34	0	2019	33120	512	5.4E+09	0	34404	523	0	14	6	6	0	22	0	0	0	16	3	0	0	4096	
malware,	15782	8192	16384	8226	37120	2071	34112	512	2.7E+08	0	332	267	0	48	4	4	0	0	0	0	0	16	3	0	0	8192	
malware,	5344	4096	0	559	0	2019	0	512	4194304	0	34404	523	0	2	4	5	0	30	0	2	16	9	0	0	4096		
malware,	4128736	3592192	0	34	0	2019	33120	512	5.4E+09	0	34404	523	0	14	6	6	0	22	0	0	0	16	3	0	0	4096	
malware,	4128736	3592192	0	34	0	2019	33120	512	5.4E+09	0	34404	523	0	14	6	6	0	22	0	0	0	16	3	0	0	4096	
malware,	4124400	3588096	0	34	0	2019	33120	512	5.4E+09	0	34404	523	0	14	6	6	0	22	0	0	0	16	3	0	0	4096	
malware,	4128736	3592192	0	34	0	2019	33120	512	5.4E+09	0	34404	523	0	14	6	6	0	22	0	0	0	16	3	0	0	4096	

Hình 3.1: Bản ghi dữ liệu sau khi thực hiện trích xuất và gán nhãn

Chuẩn hóa dữ liệu

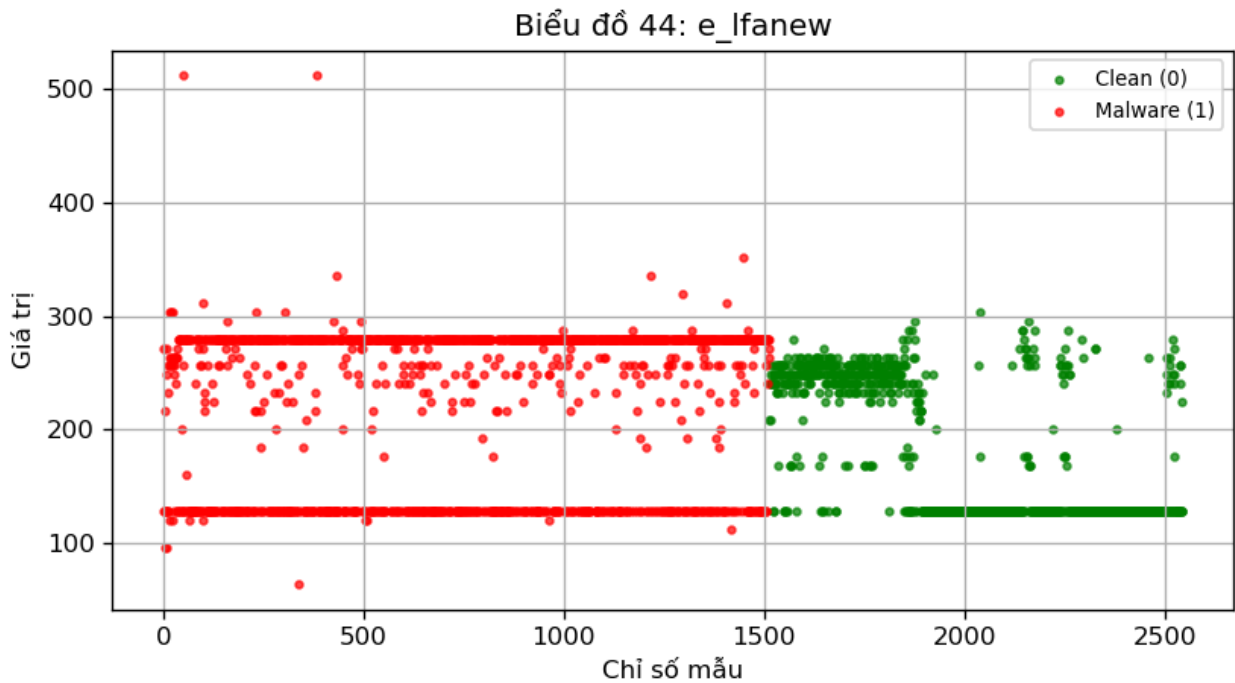
Không biến đổi dữ liệu , sử dụng dữ liệu thô để tiến hành mô hình.

Khảo sát và lựa chọn đặc trưng cho mô hình học máy

Tiến hành phân tích, thống kê toàn bộ tập dữ liệu khảo sát với 2.078 bản ghi (1.046 bản ghi đầu là mã độc, 1.032 bản ghi sau là sạch) để trích chọn đặc trưng cho mô hình học máy. Trong đó, lựa chọn các đặc trưng có giá trị phân biệt giữa mã độc và file sạch cao, còn các đặc trưng có giá trị phân biệt thấp hoặc không đổi cần loại bỏ.

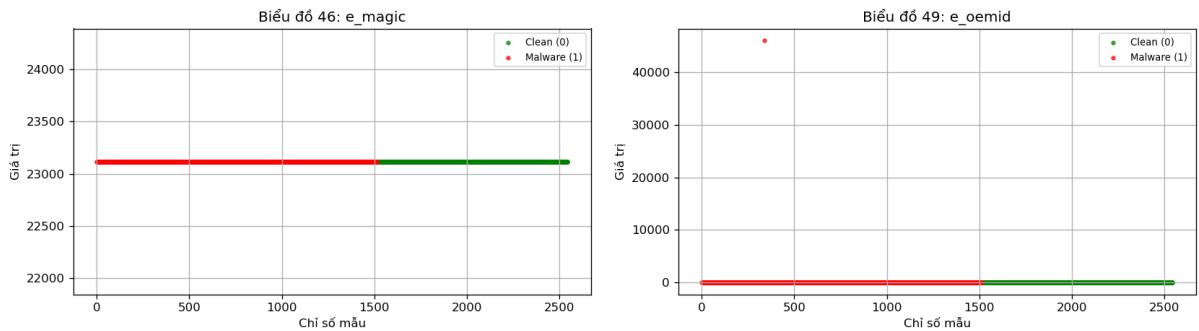
a) Thống kê lựa chọn trong 19 đặc trưng của IMAGE_DOS_HEADER:

Chọn 1 đặc trưng có giá trị phân biệt cao trên đồ thị nửa đầu (mã độc) và nửa cuối (file sạch) là “e_lfanew”.



Hình 3.2: Đặc trưng e_lfanew

Các đặc trưng còn lại có giá trị phân biệt kém hoặc không đổi, cần loại bỏ.

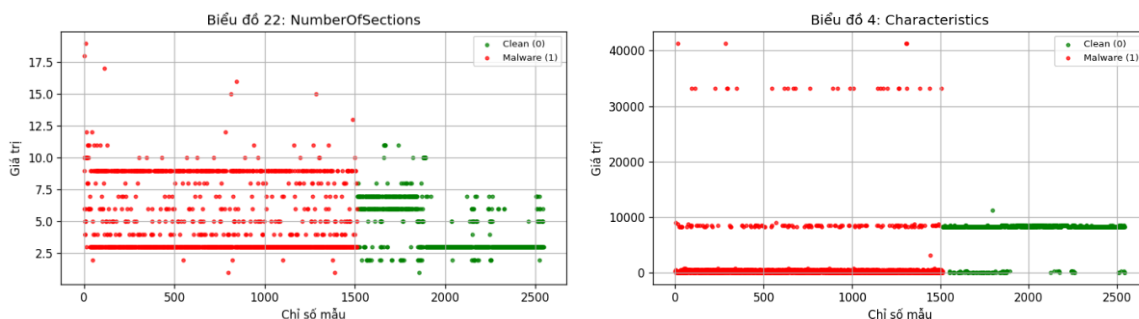


Hình 3.3: Đặc trưng e_magic và đặc trưng e_crhc

b) Khảo sát thống kê lựa chọn trong 7 đặc trưng của IMAGE_FILE_HEADER bằng cách khảo sát tương tự như mục (a):

- Chọn 2 đặc trưng có giá trị phân biệt cao là “NumberOfSection” và “Characteristics”.

- Số các đặc trưng còn lại có giá trị phân biệt kém hoặc không đổi cần loại bỏ.

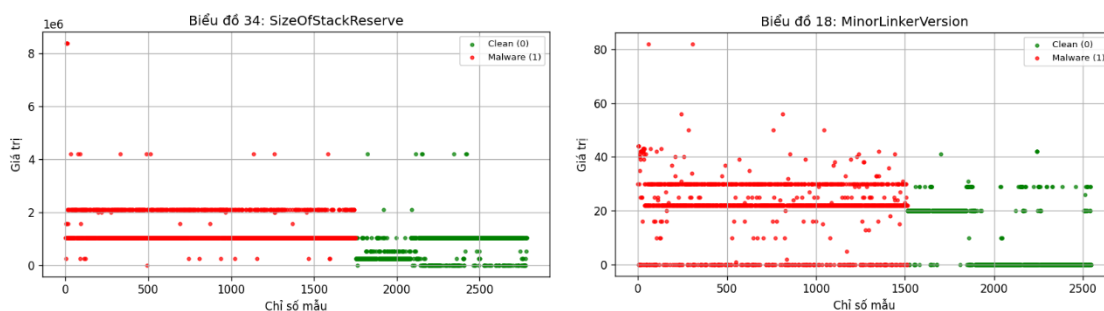


Hình 3.4: Đặc trưng NumberOfSection và Characteristics

c) Khảo sát thống kê IMAGE_OPTIONAL_HEADER và lựa chọn trong 29 đặc trưng: thống kê tương tự như các bước (a),(b) ta thu được:

Chọn 11 đặc trưng có giá trị phân biệt cao:

“BaseOfCode”, “SizeOfHeader”, “SizeOfImage”, “SizeOfUninitializedData”, “SectionAlignment”, “MinorLinkerVersion”, “MajorLinkerVersion”, “SizeOfCode”, “AddressOfEntryPoint”, “MajorSubsystemVersion”, “SizeOfStackReserve”



Hình 3.5: Hai đặc trưng SizeOfStackReverse và MinorLinkerVersion

Số các đặc trưng còn lại có giá trị phân biệt kém hoặc không đủ cần được loại bỏ. Cuối cùng sau khi phân tích thống kê, ta trích chọn 14 đặc trưng:

TT	Đặc trưng	TT	Đặc trưng
1	E_lfanew	8	SizeOfUninitializedData
2	Characteristic	9	SizeOfHeader
3	NumberOfSections	10	AddressOfEntrypoint
4	MajorSubsystemVersion	11	BaseOfCode
5	MinorLinkerVersion	12	SizeOfStackReserve
6	MajorLinkerVersion	13	SizeOfCode
7	SectionAlignment	14	SizeOfImage

3.2 Cài đặt thử nghiệm

3.2.1 Môi trường cài đặt

Công cụ được cài đặt trên môi trường hệ điều hành Windows 10 máy có cấu hình như sau:

Phần cứng	Phần mềm và môi trường phát triển
CPU Core(TM) i7-11800H @ 2.3GHz	HĐH: Windows 10
RAM 16GB	IDE: Spyder Anaconda
SSD 512GB	Môi trường: Python3
	Thư viện: Sklearn, Pickle, Pefile

Yêu cầu các phần mềm hỗ trợ:

STT	Tên	Giấy phép	Địa chỉ tải
1	Python 3.8	Gpl	https://www.python.org/
2	Anaconda	Anaconda Inc	https://repo.anaconda.com/archive/Anaconda3-2021.05-Windows-x86_64.exe

3.2.2 Thử nghiệm

Các bước thực hiện:

Bước 1: Chạy file Extractor.py với các file pe tìm được thu được malware.csv và benign.csv

Bước 2: Chạy file Combine.py với malware.csv và benign.csv thu được dataset.csv

Bước 3: Chạy file Graph.py với dataset.csv

Bước 4: Chạy file Feature_selector.py với Dataset.csv thu được file Dataset_selected.csv

Bước 5: Chạy file Model_training.py lần lượt với file Dataset.csv và file Dataset_selected.csv

3.3 Đánh giá kết quả

Đánh giá Confusions Matrix

Cách đơn giản và hay được sử dụng nhất là Accuracy (độ chính xác). Cách đánh giá này đơn giản tính tỉ lệ giữa số điểm được dự đoán đúng và tổng số điểm trong tập dữ liệu kiểm thử

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

Đánh giá Precision và Recall

Trong thực nghiệm đã thực hiện, số lượng các lớp không đồng đều, có chút chênh lệch. Với những trường hợp như vậy, phép đo khá hiệu quả được sử dụng là Precision – Recall

$$\text{Precision} = \frac{TP}{TP + FP}; \quad \text{Recall} = \frac{TP}{TP + FN}$$

Precision và Recall đều là các số không âm nhỏ hơn hoặc bằng một. Precision cao đồng nghĩa với việc độ chính xác của các điểm tìm được là cao. Recall cao đồng nghĩa với việc True Positive Rate cao, tức tỉ lệ bỏ sót các điểm thực sự positive là thấp. Từ hai đại lượng tính được ta có thể thấy mô hình phân lớp triển khai được là rất tốt.

Đánh giá F1-score

Bên cạnh Precision và Recall, chỉ số F₁-score cũng được sử dụng rộng rãi để đánh giá hiệu quả của mô hình phân loại, đặc biệt là trong các bài toán mất cân bằng

dữ liệu. F₁-score là trung bình điều hòa (*harmonic mean*) của Precision và Recall, giúp cân bằng giữa hai chỉ số này:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

F₁-score có giá trị nằm trong khoảng (0, 1]:

- Khi cả Precision và Recall đều bằng 1 → F₁ = 1 (mô hình hoàn hảo).
- Nếu một trong hai chỉ số bằng 0 → F₁ = 0 (mô hình kém).

Kết quả thử nghiệm

Mô hình	Phân lớp	P	R	F ₁
NB TGHL:0.0108s TGTN:0.0033s	Benign	1	0.10	0.19
	Malware	0.65	1	0.79
	Avg	0.83	0.55	0.49
DT TGHL:0.0180s TGTN:0.0016s	Benign	0.97	0.97	0.97
	Malware	0.98	0.98	0.98
	Avg	0.98	0.98	0.98
RF TGHL:0.3851s TGTN:0.0148s	Benign	0.98	0.99	0.98
	Malware	0.99	0.99	0.99
	Avg	0.98	0.99	0.99

Bảng 1. Kết quả phân lớp với Dataset.csv gồm 53 đặc trưng

Nhận xét:

- Naive Bayes (NB)
 - Precision cao (1.00) cho Benign, nhưng Recall cực thấp (0.10) $\Rightarrow$ Dự đoán hầu hết mẫu là Malware $\rightarrow$ báo động giả ít, nhưng bỏ sót Benign cực nhiều.
 - Ngược lại với Malware, Recall = 1.00 $\Rightarrow$ phát hiện đủ Malware, nhưng Precision = 0.65 $\Rightarrow$ nhiều mẫu Benign bị nhầm là Malware.
 - F1 avg = 0.49 $\Rightarrow$ rất thấp $\rightarrow$ Mô hình không cân bằng giữa hai lớp.
- Decision Tree (DT)
 - Precision, Recall, F1 đều từ 0.97 đến 0.98.
 - Hoạt động tốt trên cả 2 lớp Benign/Malware.
 - Huấn luyện nhanh (TGHL = 0.0180s), dự đoán cực nhanh.
- Random Forest (RF)
 - Precision & Recall đều gần 1.00 $\rightarrow$ Gần như hoàn hảo.
 - F1-score = 0.99 $\rightarrow$ rất ổn định.
 - Mất thời gian huấn luyện hơn (TGHL = 0.14s) nhưng vẫn chấp nhận được.

Kết luận:

- Đối với mô hình Decision Tree, hiệu suất rất tốt và cân bằng, F_1 cao thể hiện mô hình rất ổn định và đáng tin cậy.
- Random Forest là mô hình tốt nhất trong cả 3 mô hình, đặc biệt là tính ổn định và khả năng phát hiện mã độc lẫn file sạch, tuy thời gian huấn luyện có lớn hơn hai mô hình còn lại nhưng không đáng kể

Mô hình	Phân lớp	P	R	F_1
NB TGHL:0.0041s TGTN:0.0039s	Benign	0.67	0.99	0.88
	Malware	0.99	0.71	0.83
	Avg	0.83	0.85	0.81
DT TGHL:0.0191s TGTN:0.0045s	Benign	0.96	0.96	0.96
	Malware	0.98	0.98	0.98
	Avg	0.97	0.97	0.97
RF TGHL:0.4264s TGTN:0.0148s	Benign	0.97	0.97	0.97
	Malware	0.98	0.98	0.98
	Avg	0.98	0.98	0.98

Bảng 2. Kết quả phân lớp với Dataset_selected.csv gồm 14 đặc trưng

Nhận xét:

- Naive Bayes (NB)

- Precision cao (0.99) với lớp malware, nhưng Recall chỉ 0.71, dẫn đến việc dễ bỏ sót một số mẫu malware
- Ngược lại, lớp Benign có Precision thấp hơn (0.67), nhưng Recall khá cao (0.99)
- F_1 -score = 0.81, hiệu suất tạm ổn nhưng kém ổn định hơn so với hai mô hình còn lại
- TGHL và TGTN đều rất thấp, mô hình huấn luyện và dự đoán rất nhanh

- Decision Tree

- Cả hai lớp đều có Precision và Recall từ 0.96 - 0.98, hiệu suất rất cân bằng
- F_1 -score = 0.97, mô hình tốt và rất đáng tin cậy
- TGHL và TGTN của Decision tree cũng rất nhanh

- Random Forest
 - Precision, Recall, F_1 đều rất cao, hiệu quả phân loại rất tốt
 - Tuy nhiên, TGH là cao nhất trong ba mô hình, TGTN ở mức tạm ổn

Kết luận:

- Random Forest vẫn là mô hình mạnh nhất về độ chính xác, mặc dù thời gian huấn luyện có lâu hơn
- Decision Tree là mô hình có hiệu suất tốt và cân bằng, cho dù đã giảm số lượng đặc trưng xuống đáng kể
- Naive Bayes là mô hình có tốc độ huấn luyện và dự đoán cao, bù lại hiệu suất phân loại kém ổn định hơn
- Việc giảm số lượng đặc trưng từ 55 xuống 14 không gây ảnh hưởng lớn đến hiệu quả mô hình, cho thấy việc lựa chọn đặc trưng hợp lý có thể tối ưu hiệu suất tính toán mà không đánh đổi quá nhiều độ chính xác.

3.4 Kết luận chương 3

Chương 3 đã mô hình hóa, triển khai được thuật toán nghiên cứu được từ chương 2. Từ mô hình triển khai này, đánh giá được kết quả nghiên cứu, tính ứng dụng và khả năng mở rộng của bài báo cáo.

KẾT LUẬN

Ba chương của bài báo cáo đã thể hiện được rằng nhưng mục tiêu đặt ra khi thực hiện bài báo cáo đều đã đạt được. Cụ thể:

Chương 1 trình bày được bức tranh tổng quan của các phương pháp phân tích mã độc hiện tại. Qua đó thấy được ưu nhược điểm của các phương pháp này. Và tầm quan trọng của giải pháp phát hiện mẫu trước khi phân tích.

Chương 2 giải thích được khái niệm siêu dữ liệu, cấu trúc tệp tin thực thi trên hệ điều hành Windows, các trường quan trọng được sử dụng làm siêu dữ liệu. Bên cạnh đó còn trình bày về một số mô hình phân lớp dữ liệu. Đưa ra những khái niệm và thuật toán liên quan. Từ những ví dụ và những công thức tường minh, nội dung chương đã nêu ra được cơ sở khoa học cho nghiên cứu, tạo giải pháp giải quyết bài toán đặt ra

Chương 3 đã hiện thực hóa được mô hình và cơ sở khoa học đã tìm hiểu được, mô hình hóa được bài toán cụ thể cần giải quyết và một giải pháp mang hướng có thể triển khai thực tế

Trong quá trình làm bài báo cáo bọn em đã thu được rất nhiều kiến thức về cấu trúc tệp tin thực thi trên hệ điều hành Windows, cô đọng lại kiến thức về mã độc, tiếp cận được nhiều thuật toán khai phá dữ liệu với hướng tiếp cận cây quyết định. Hiểu được các đề phát hiện dữ liệu bằng học máy, hiểu được quy trình triển khai mô hình học máy. Hiểu được tầm quan trọng của việc phát hiện dữ liệu, nhất là đối với dữ liệu là mã độc. Ngoài ra thu được rất nhiều kiến thức thực tế khi triển khai mô hình phát hiện mã độc.

Bên cạnh đó là những khó khăn, hạn chế trong quá trình thực hiện nghiên cứu vào hoàn thiện bài báo cáo.

Vì giới hạn về thời gian nghiên cứu, lượng dữ liệu, mô hình tiếp cận được còn ít, năng lực bản thân hiện tại còn rất giới hạn về kiến thức và kinh nghiệm. Nhưng thay vào đó đề tài đưa ra là một vấn đề rất thực tế, là xu hướng hiện tại, có bước dài về hướng phát triển nên nếu có thể bọn em sẽ cố gắng nghiên cứu sâu hơn về học máy, tiếp cận nhiều hơn với mô hình tối ưu hơn, xây dựng GUI giao diện đồ họa cho ứng dụng, xây dựng cơ sở dữ liệu có phân quyền, bảo vệ dữ liệu và rất nhiều tham vọng khác về hướng phát triển nghiên cứu. Ngoài ra, hiện tại ở Việt Nam các công ty bảo mật hàng đầu cũng đang chạy đua để tích hợp trí tuệ nhân tạo, học máy vào hệ thống phát hiện, xử lý mã độc nên nếu nghiên cứu có thể phát triển tốt, bọn em hy vọng nó sẽ là một module quan trọng với ý nghĩa phát hiện mã độc dựa trên kỹ thuật học máy, trí tuệ nhân tạo.

TÀI LIỆU THAM KHẢO

- [1] D. Gilbert, C. Mateu, and J. Planes, “The rise of machine learning for detection and classification of malware: Research developments, trends and challenges,” *J. Netw. Comput. Appl.*, vol. 153, p. 102526, 2020.
- [2] K. Chumachenko, “Machine Learning Methods for Malware Detection and Classification,” *Cuckoo Sandbox*, 2017.
- [3] J. Aycock, *Computer Viruses and Malware*. Springer, 2006.
- [4] Tạp chí An toàn thông tin, “Phát hiện mã độc dựa vào học máy và thông tin PE Header (Phần I),” *An Toàn Thông Tin*, [Trực tuyến]. Có tại: [Phát hiện mã độc dựa vào học máy và thông tin PE Header \(Phần I\) | An toàn thông tin](#)
- [5] U. Fatima, M. Ali, N. Ahmed, and M. Rafiq, “Numerical modelling of susceptible latent breaking-out quarantine computer virus epidemic dynamics,” *Appl. Math.*, 2018.
- [6] Z.-H. Zhou, *Ensemble Methods: Foundation and Algorithm*. Springer.
- [7] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning*. Springer.
- [8] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*. Springer.
- [9] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Chapman & Hall, 1993.
- [10] M. Sikorski and A. Honig, *Practical Malware Analysis*. No Starch Press, 2012.

- [11] J. Furner, “Definitions of ‘Metadata’: A brief survey of international standards,” *J. Assoc. Inf. Sci. Technol.*, 2019.
- [12] J.-R. Park and E. Childress, “Dublin Core metadata semantics: An analysis of the perspectives of information professionals,” *J. Inf. Sci.*, 2009.
- [13] M. Belaoued and S. Mazouzi, “A real-time PE-malware detection system based on Chi-square test and PE-file features,” *Comput. Sci. Appl.*, 2021.
- [14] J. R. Quinlan, “Introduction of Decision Trees,” *Comput. Sci., Mach. Learn.*, 2004.
- [15] Y. Lemmou, J.-L. Lanet, and E. M. Souidi, “A behavioural in-depth analysis of ransomware infection,” *Original Res. Paper*, 2022.
- [16] D. Moore, C. Shannon, and K. Claffy, “Code-Red: A case study on the spread and victims of an Internet worm,” in *Proc. 2nd ACM SIGCOMM Workshop Internet Measurement*, 2002.